



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Lógica Digital

Circuitos combinacionales

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

#	Descripción	Axioma	Dual
A1	Existen dos elementos	$B = \{0, 1\}$	
A2	operador negación:	$\bar{0} = 1$	$\bar{1} = 0$
A3	<i>idempotencia</i>	$0 \cdot 0 = 0$	$1 + 1 = 1$
A4		$1 \cdot 1 = 1$	$0 + 0 = 0$
A5	Elemento dominante	$0 \cdot 1 = 1 \cdot 0 = 0$	$0 + 1 = 1 + 0 = 1$

De los axiomas anteriores se derivan las siguientes propiedades:

[illegible]

De los axiomas anteriores se derivan las siguientes propiedades:

[illegible]

De los axiomas anteriores se derivan las siguientes propiedades:

[illegible]

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$
Asociatividad	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$
Asociatividad	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B \cdot C) = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$
Asociatividad	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B \cdot C) = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$
Absorción	$A \cdot (A + B) = A$	$A + A \cdot B = A$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$
Asociatividad	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B \cdot C) = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$
Absorción	$A \cdot (A + B) = A$	$A + A \cdot B = A$
De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

De los axiomas anteriores se derivan las siguientes propiedades:

Propiedad	AND	OR
Identidad	$1 \cdot A = A$	$0 + A = A$
Nulo	$0 \cdot A = 0$	$1 + A = 1$
Idempotencia	$A \cdot A = A$	$A + A = A$
Inverso	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Conmutatividad	$A \cdot B = B \cdot A$	$A + B = B + A$
Asociatividad	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributividad	$A + (B \cdot C) = (A + B) \cdot (A + C)$	$A \cdot (B + C) = A \cdot B + A \cdot C$
Absorción	$A \cdot (A + B) = A$	$A + A \cdot B = A$
De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$

Tarea: ¡Demostrarlas!

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot \overline{Y} \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot \overline{Y} \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot \overline{Y} \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$\overline{\overline{X} \cdot \overline{Y} \cdot Z} + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{De Morgan}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{De Morgan}$$

$$(X + \overline{Y}) \cdot Z + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{De Morgan}$$

$$(X + \overline{Y}) \cdot Z + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$(X + \overline{Y}) \cdot (Z + \overline{Z}) \quad \leftarrow \text{Inverso}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{De Morgan}$$

$$(X + \overline{Y}) \cdot Z + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$(X + \overline{Y}) \cdot (Z + \overline{Z}) \quad \leftarrow \text{Inverso}$$

$$(X + \overline{Y}) \cdot 1 \quad \leftarrow \text{Identidad}$$

Demostrar si la siguiente igualdad entre funciones booleanas es verdadera o falsa:

$$(X + \overline{Y}) = \overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)}$$

Solución:

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{(Y + Z)} \quad \leftarrow \text{De Morgan}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + X \cdot \overline{Z} + \overline{Y} \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$\overline{\overline{X} \cdot Y \cdot Z} + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{De Morgan}$$

$$(X + \overline{Y}) \cdot Z + (X + \overline{Y}) \cdot \overline{Z} \quad \leftarrow \text{Distributiva}$$

$$(X + \overline{Y}) \cdot (Z + \overline{Z}) \quad \leftarrow \text{Inverso}$$

$$(X + \overline{Y}) \cdot 1 \quad \leftarrow \text{Identidad}$$

$$X + \overline{Y} \leftarrow \text{Lo que queríamos demostrar.}$$

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

$$AB \equiv A \cdot B \equiv A \text{ AND } B$$

En el lenguaje coloquial vamos a llamar a las operaciones indistintamente de la siguiente forma:

$$A + B \equiv A \text{ OR } B$$

$$AB \equiv A \cdot B \equiv A \text{ AND } B$$

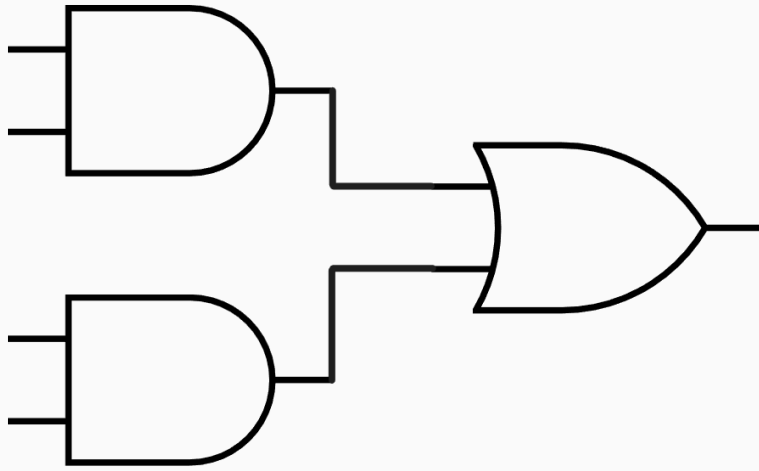
$$\overline{A} \equiv \text{NOT } A$$

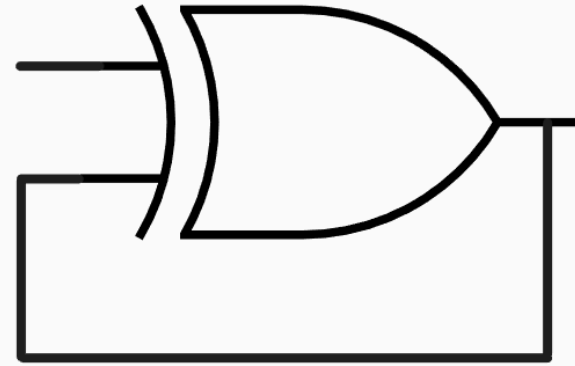
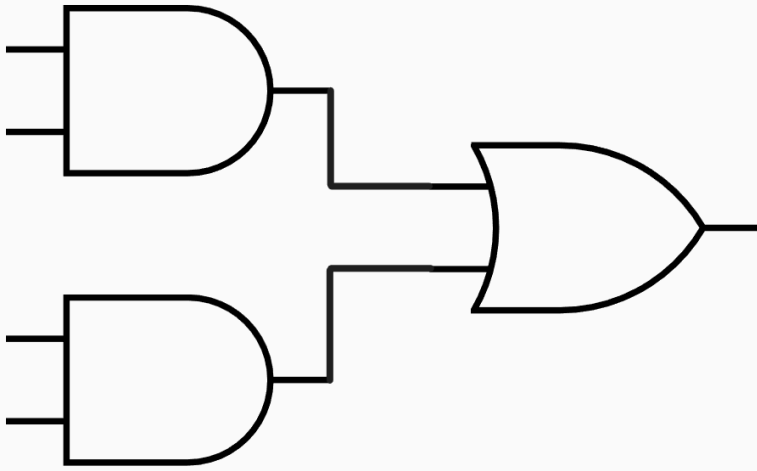
Un sistema digital puede ser:

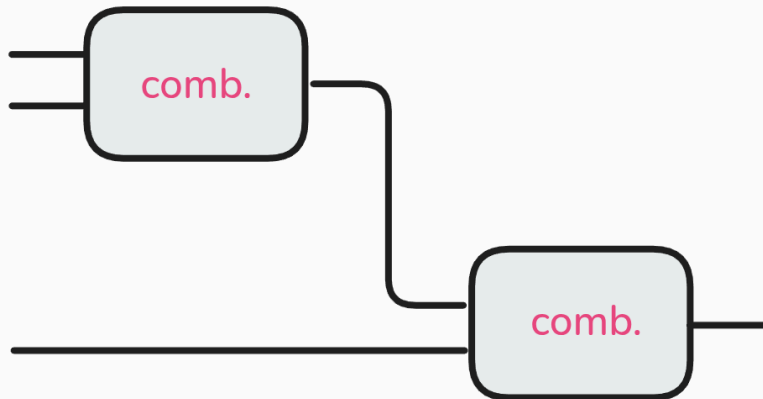
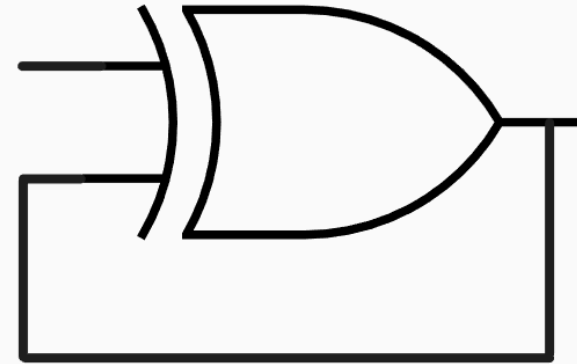
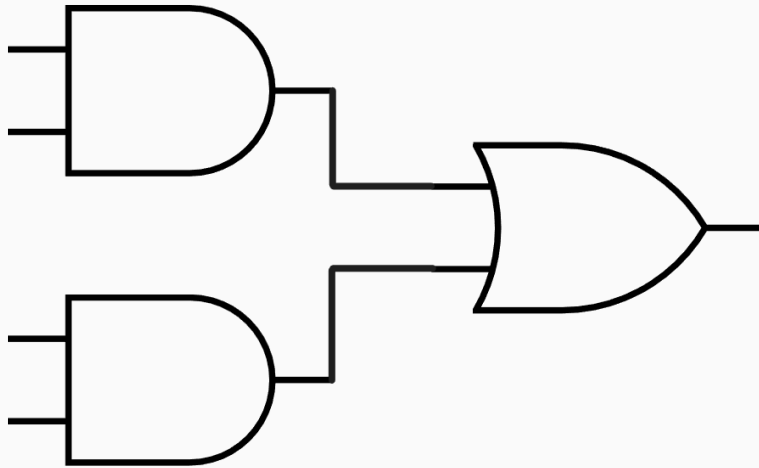
- **Combinacional:** si sus salidas dependen únicamente de las entradas actuales.

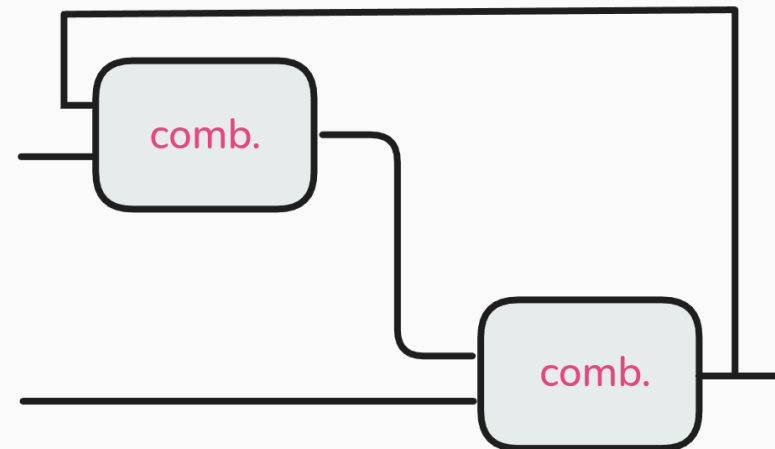
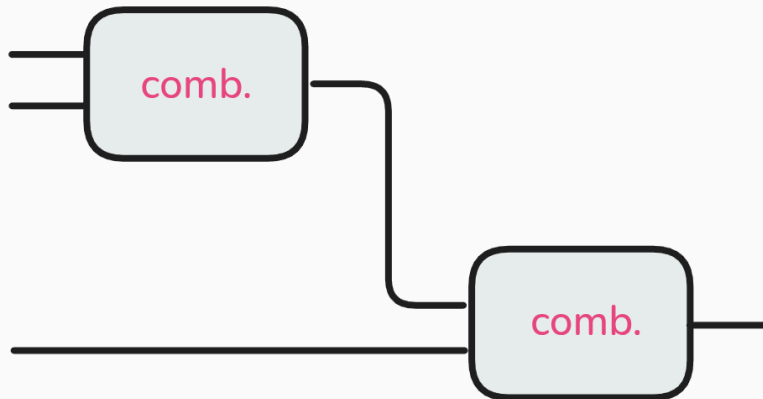
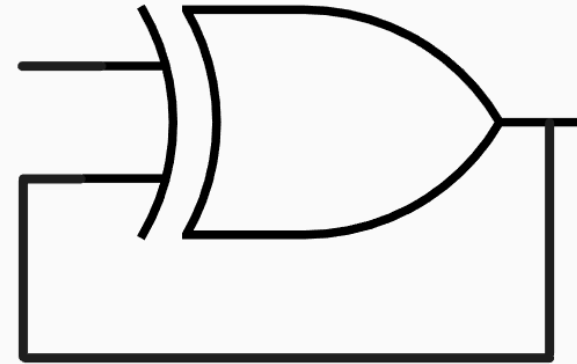
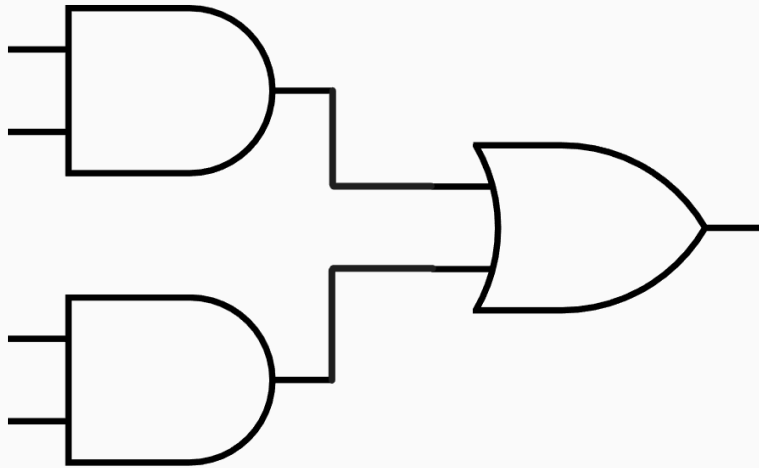
Un sistema digital puede ser:

- **Combinacional:** si sus salidas dependen únicamente de las entradas actuales.
- **Secuencial:** si sus salidas dependen de las entradas actuales y de entradas anteriores (tiene memoria).







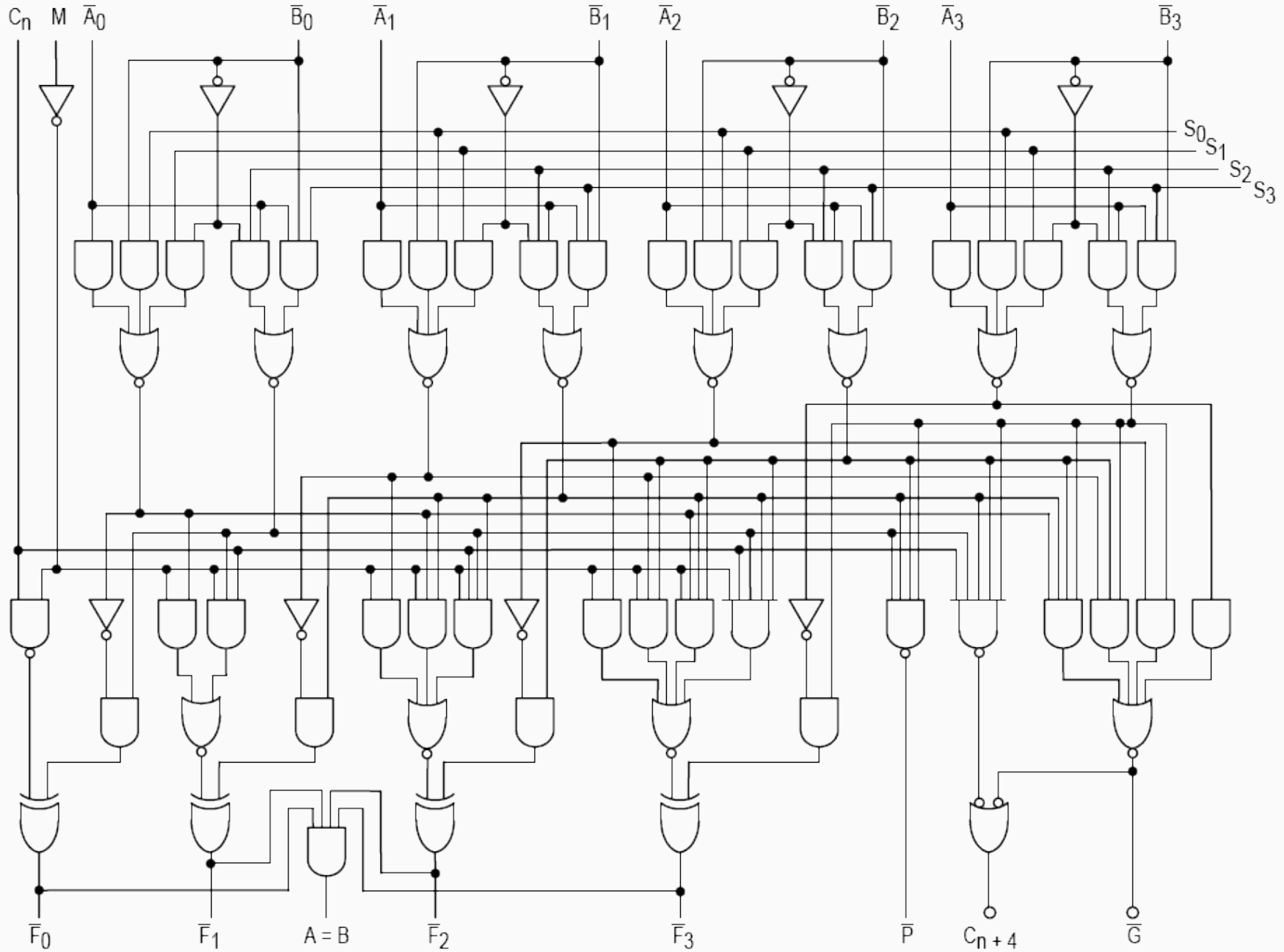


Entradas y salidas - Categorización

Por momentos vamos a querer abstraer nuestros circuitos en módulos de los cuales observaremos solamente sus entradas y salidas.

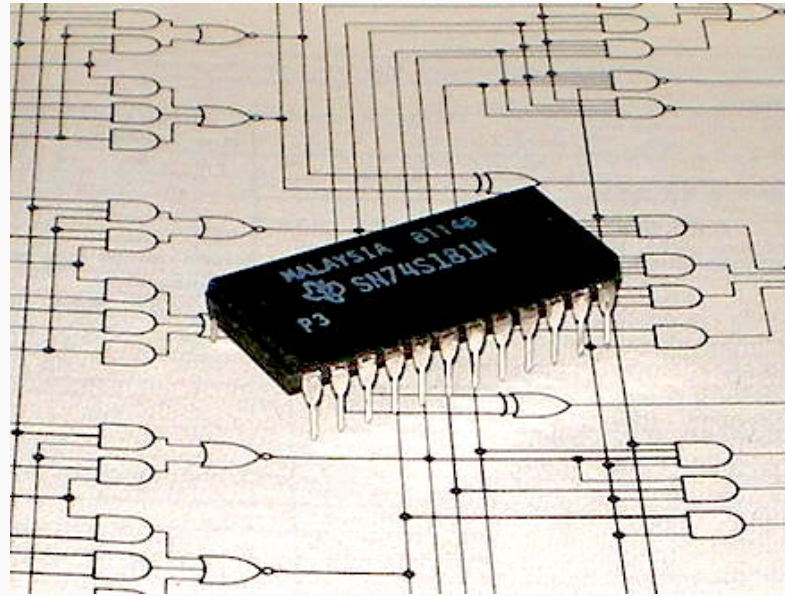
Veamos un ejemplo donde ocultamos parte de la complejidad pasando de una vista interna del circuito (caja blanca) a una externa (caja negra).

Desde el museo: ALU 74181

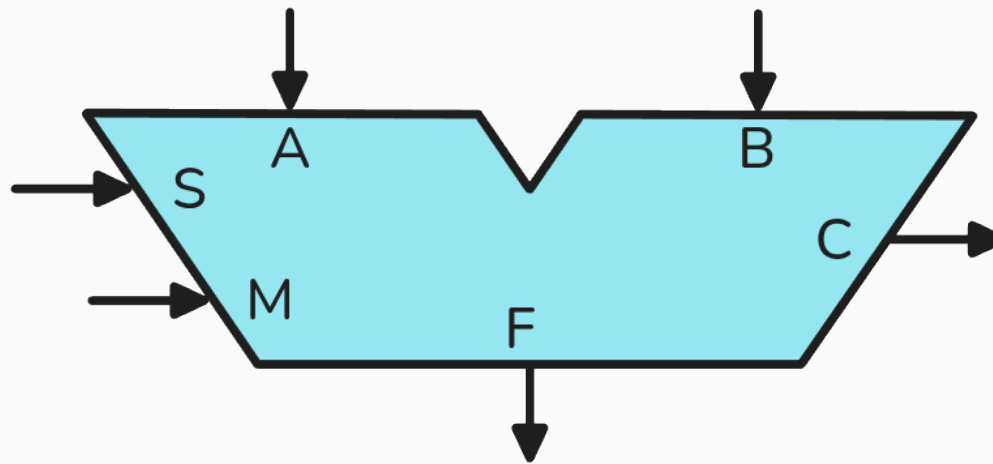


- La primera ALU completa de 4 bits integrada en un solo chip.
- 16 funciones lógicas y 16 funciones aritméticas.
- Podía realizar todas las operaciones tradicionales: suma, resta, decremento, con o sin acarreo, así como AND, NAND, OR, NOR, XOR y shift, entre otras.

- La primera ALU completa de 4 bits integrada en un solo chip.
- 16 funciones lógicas y 16 funciones aritméticas.
- Podía realizar todas las operaciones tradicionales: suma, resta, decremento, con o sin acarreo, así como AND, NAND, OR, NOR, XOR y shift, entre otras.



Aplicando lo anterior, podemos trabajar con la ALU viéndola de la siguiente manera:



Establecen el sentido de la información:

En la ALU anterior se representan con las flechas...

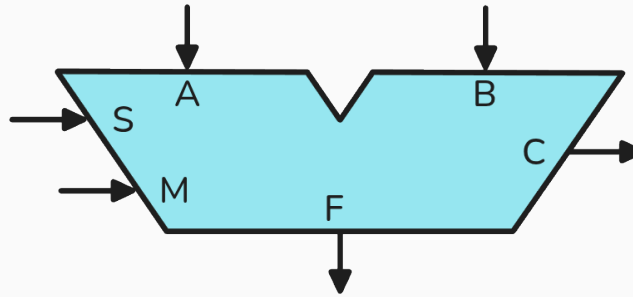
Establecen el sentido de la información:

En la ALU anterior se representan con las flechas...

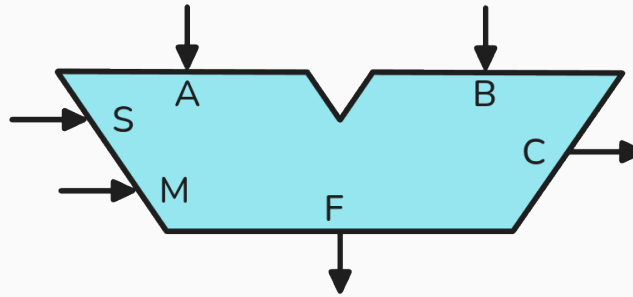
```
module ALU #(parameter WIDTH = 4)
(
    input [WIDTH-1:0] A,
    input [WIDTH-1:0] B,
    input [3:0] op,
    output [WIDTH-1:0] result,
    output carry_out
);
endmodule
```

SV

En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?



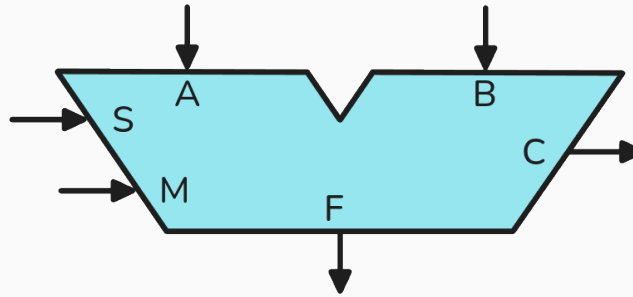
En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?



NO

Datos vs. Control

En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?

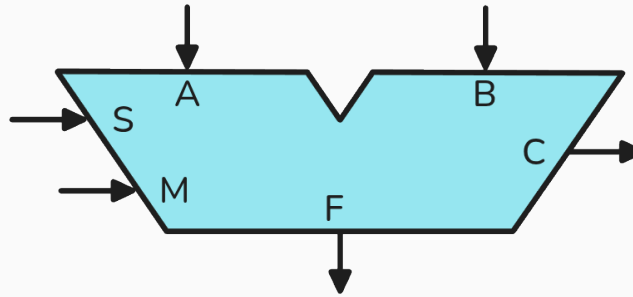


NO

Datos vs. Control

Datapath: conjunto de señales que transportan datos entre los distintos bloques de un sistema digital.

En la ALU, ¿son funcionalmente todas iguales las entradas y salidas?



NO

Datos vs. Control

Datapath: conjunto de señales que transportan datos entre los distintos bloques de un sistema digital.

Control path: conjunto de señales que controlan el comportamiento del sistema digital.

Lógica proposicional a circuitos combinacionales



El estudio de la lógica proposicional y del álgebra de Boole tiene que ver con que vamos a querer implementar funciones lógicas utilizando sistemas combinacionales.

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica

$$Y = f(A, B)$$

tal que:

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica

$$Y = f(A, B)$$

tal que:

A	B	Y	minitérmino	nombre
0	0	0		
0	1	1		
1	0	0		
1	1	0		

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica

$$Y = f(A, B)$$

tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	
0	1	1	$\overline{A} \cdot B$	
1	0	0	$A \cdot \overline{B}$	
1	1	0	$A \cdot B$	

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica

$$Y = f(A, B)$$

tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	0	$A \cdot B$	m_3

La primera forma que vamos a ver es la llamada **suma de productos** (o *Sum of Products*, **SOP**).

Supongamos que queremos implementar la función lógica

$$Y = f(A, B)$$

tal que:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	0	$A \cdot B$	m_3

$$Y = f(A, B) = \overline{A} \cdot B = m_1$$

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	1	$A \cdot B$	m_3

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	1	$A \cdot B$	m_3

$$Y = f(A, B) = \overline{A} \cdot B + A \cdot B = m_1 + m_3$$

Y si tenemos más de un 1 en la salida de la tabla de verdad, por ejemplo:

A	B	Y	minitérmino	nombre
0	0	0	$\overline{A} \cdot \overline{B}$	m_0
0	1	1	$\overline{A} \cdot B$	m_1
1	0	0	$A \cdot \overline{B}$	m_2
1	1	1	$A \cdot B$	m_3

$$Y = f(A, B) = \overline{A} \cdot B + A \cdot B = m_1 + m_3$$

$$Y = \sum m(1, 3)$$

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = f(A, B, C) = \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C = m_0 + m_4 + m_5$$

Y si tenemos más de 2 variables:

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = f(A, B, C) = \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot C = m_0 + m_4 + m_5$$

$$Y = \sum m(0, 4, 5)$$

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	0	$\overline{A} + \overline{B}$	M_3

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	0	$\overline{A} + \overline{B}$	M_3

$$Y = f(A, B) = (A + B) \cdot (\overline{A} + B) \cdot (\overline{A} + \overline{B}) = M_0 \cdot M_2 \cdot M_3$$

La segunda forma que vamos a ver es la llamada **producto de sumas (POS)**.

A	B	Y	maxitérmino	nombre
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	0	$\overline{A} + \overline{B}$	M_3

$$Y = f(A, B) = (A + B) \cdot (\overline{A} + B) \cdot (\overline{A} + \overline{B}) = M_0 \cdot M_2 \cdot M_3$$

$$Y = \prod M(0, 2, 3)$$

Circuitos básicos



Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

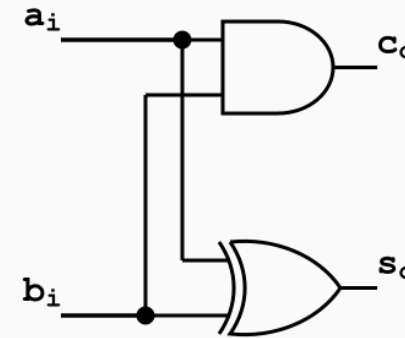
Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementación:



Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

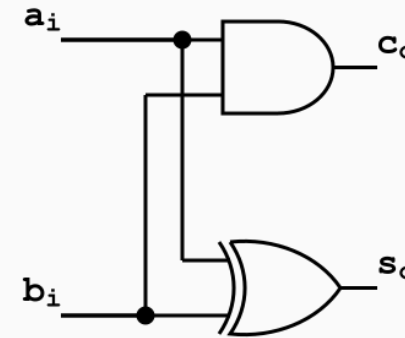
Sumador simple (*half-adder*)

Suma dos bits y produce la suma y el acarreo.

Tabla de verdad:

a_i	b_i	s_o	c_o
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementación:



Ecuaciones lógicas:

$$s_o = (a_i \cdot \overline{b_i}) + (\overline{a_i} \cdot b_i)$$

$$= a_i \oplus b_i$$

$$c_o = a_i \cdot b_i$$

```
module half_adder (SV
    input logic a_i,
    input logic b_i,
    output logic s_o,
    output logic c_o
);

    assign s_o = a_i ^ b_i;
    assign c_o = a_i & b_i;

endmodule
```

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ecuaciones lógicas:

$$\begin{aligned}s_o &= (\overline{c_i} \cdot \overline{a_i} \cdot b_i) + (\overline{c_i} \cdot a_i \cdot \overline{b_i}) \\ &\quad + (c_i \cdot \overline{a_i} \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \oplus b_i) \oplus c_i \\ c_o &= (\overline{c_i} \cdot a_i \cdot b_i) + (c_i \cdot \overline{a_i} \cdot b_i) \\ &\quad + (c_i \cdot a_i \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \cdot b_i) + (c_i \cdot (a_i \oplus b_i))\end{aligned}$$

Sumador completo (*full-adder*)

Suma dos bits y un acarreo de entrada, y produce la suma y el acarreo de salida.

Tabla de verdad:

c_i	a_i	b_i	s_o	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ecuaciones lógicas:

$$\begin{aligned}s_o &= (\overline{c_i} \cdot \overline{a_i} \cdot b_i) + (\overline{c_i} \cdot a_i \cdot \overline{b_i}) \\ &\quad + (c_i \cdot \overline{a_i} \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \oplus b_i) \oplus c_i \\ c_o &= (\overline{c_i} \cdot a_i \cdot b_i) + (c_i \cdot \overline{a_i} \cdot b_i) \\ &\quad + (c_i \cdot a_i \cdot \overline{b_i}) + (c_i \cdot a_i \cdot b_i) \\ &= (a_i \cdot b_i) + (c_i \cdot (a_i \oplus b_i))\end{aligned}$$

Ya se pone complicado...

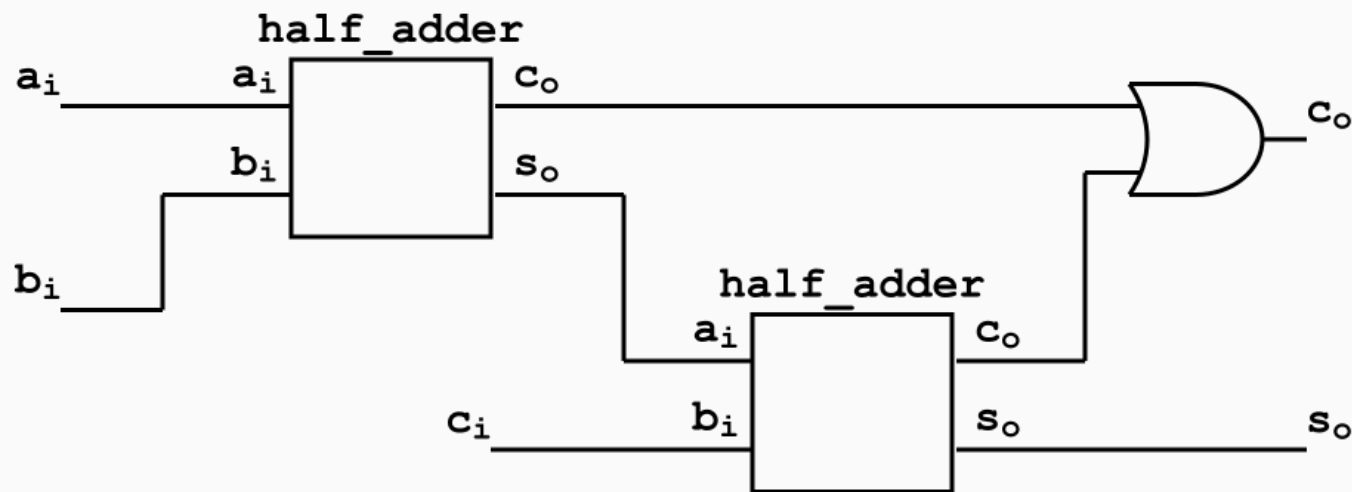
Sumador completo (*full-adder*)

Usemos lógica composicional...

Sumador completo (*full-adder*)

Usemos lógica composicional...

Ya tenemos el sumador simple, entonces:



Sumador completo (*full-adder*)

0 en SystemVerilog:

```
// Full adder using two half adders
```

SV

```
module full_adder (  
    input logic a_i,  
    input logic b_i,  
    input logic c_i,  
    output logic s_o,  
    output logic c_o  
);
```

```
    logic s1, c1, c2;
```

```
    half_adder ha1 (  
        .a_i(a_i),  
        .b_i(b_i),  
        .s_o(s1),  
        .c_o(c1)  
    );
```

```
half_adder ha2 (  
    .a_i(s1),
```

SV

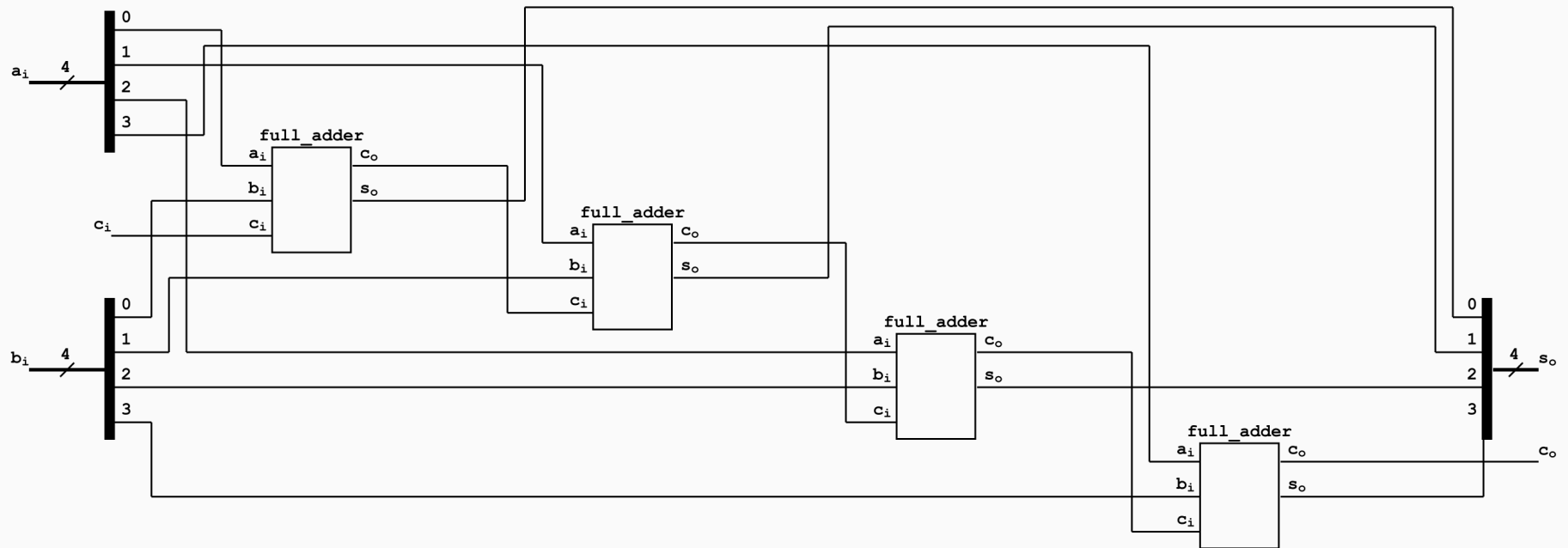
```
    .b_i(c_i),  
    .s_o(s_o),  
    .c_o(c2)  
);
```

```
    assign c_o = c1 | c2;
```

```
endmodule
```


Sumador de 4 bits (del tipo *ripple-carry*)

Ahora podemos escalar... :D



Sumador de 4 bits (del tipo *ripple-carry*)

```
// Full adder using two half adders
```

SV

```
module ripple_carry_adder
```

```
#(parameter N=4) (
```

```
    input  logic [N-1:0] a_i,
```

```
    input  logic [N-1:0] b_i,
```

```
    input  logic          c_i,
```

```
    output logic [N-1:0] s_o,
```

```
    output logic          c_o
```

```
);
```

```
    logic [N:0] c_internal; // N+1 carry bits, c_internal[0]  
    is c_i
```

```
    assign c_internal[0] = c_i;
```

```
    genvar i;
```

Sumador de 4 bits (del tipo *ripple-carry*)

```
generate
    for (i = 0; i < N; i = i + 1)
    begin : fa_gen
        full_adder fa_inst (
            .a_i(a_i[i]),
            .b_i(b_i[i]),
            .c_i(c_internal[i]),
            .s_o(s_o[i]),
            .c_o(c_internal[i+1])
        );
    end
endgenerate

assign c_o = c_internal[N];

endmodule
```

Shifter



Armar un circuito combinacional que tome un bus de 3 *bits* y lo desplace una posición hacia la izquierda o hacia la derecha, descartando el bit que “sale” por un extremo y completando con 0 el que “entra” por el otro. La dirección la elige un bit de control aparte, que no forma parte del bus de datos.

Formalmente, un shift *izq-der* de k bits tiene:

- una entrada de datos $e = (e_{k-1}, \dots, e_0)$ de k bits
- un bit de control lr
- una salida $s = (s_{k-1}, \dots, s_0)$ de k bits, definido así:
 - Si $lr = 0$ (desplazar a la **izquierda**): cada salida toma el valor de la entrada inmediata inferior, $s_i = e_{i-1}$ para $1 \leq i \leq k-1$, y entra un 0 por la derecha: $s_0 = 0$.
 - Si $lr = 1$ (desplazar a la **derecha**): cada salida toma el valor de la entrada inmediata superior, $s_i = e_{i+1}$ para $0 \leq i \leq k-2$, y entra un 0 por la izquierda: $s_{k-1} = 0$.

Ejemplos:

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

$$\text{shift_lr}(1, 011) = 001$$

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

$$\text{shift_lr}(0, 100) = 000$$

$$\text{shift_lr}(1, 011) = 001$$

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

$$\text{shift_lr}(0, 100) = 000$$

$$\text{shift_lr}(1, 011) = 001$$

$$\text{shift_lr}(0, 101) = 010$$

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

$$\text{shift_lr}(1, 011) = 001$$

$$\text{shift_lr}(0, 100) = 000$$

$$\text{shift_lr}(0, 101) = 010$$

- Si $lr = 0$, entonces $s_i = e_{i-1}$ para todo $1 \leq i \leq k-1$ y $s_0 = 0$
- Si $lr = 1$, entonces $s_i = e_{i+1}$ para todo $0 \leq i \leq k-2$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{cases} 0 & \text{si } lr=1 \\ e_1 & \text{si } lr=0 \end{cases}$$

$$\overline{lr} \cdot e_1$$

$$s_0 = \begin{cases} 0 & \text{si } lr=0 \\ e_1 & \text{si } lr=1 \end{cases}$$

$$lr \cdot e_1$$

$$s_1 = \begin{cases} e_0 & \text{si } lr=0 \\ e_2 & \text{si } lr=1 \end{cases}$$

Ejemplos:

$$\text{shift_lr}(0, 011) = 110$$

$$\text{shift_lr}(0, 100) = 000$$

$$\text{shift_lr}(1, 011) = 001$$

$$\text{shift_lr}(0, 101) = 010$$

- Si $lr = 0$, entonces $s_i = e_{i-1}$ para todo $1 \leq i \leq k-1$ y $s_0 = 0$
- Si $lr = 1$, entonces $s_i = e_{i+1}$ para todo $0 \leq i \leq k-2$ y $s_{k-1} = 0$

Solución:

$$s_2 = \begin{cases} 0 & \text{si } lr=1 \\ e_1 & \text{si } lr=0 \end{cases}$$

$$\overline{lr} \cdot e_1$$

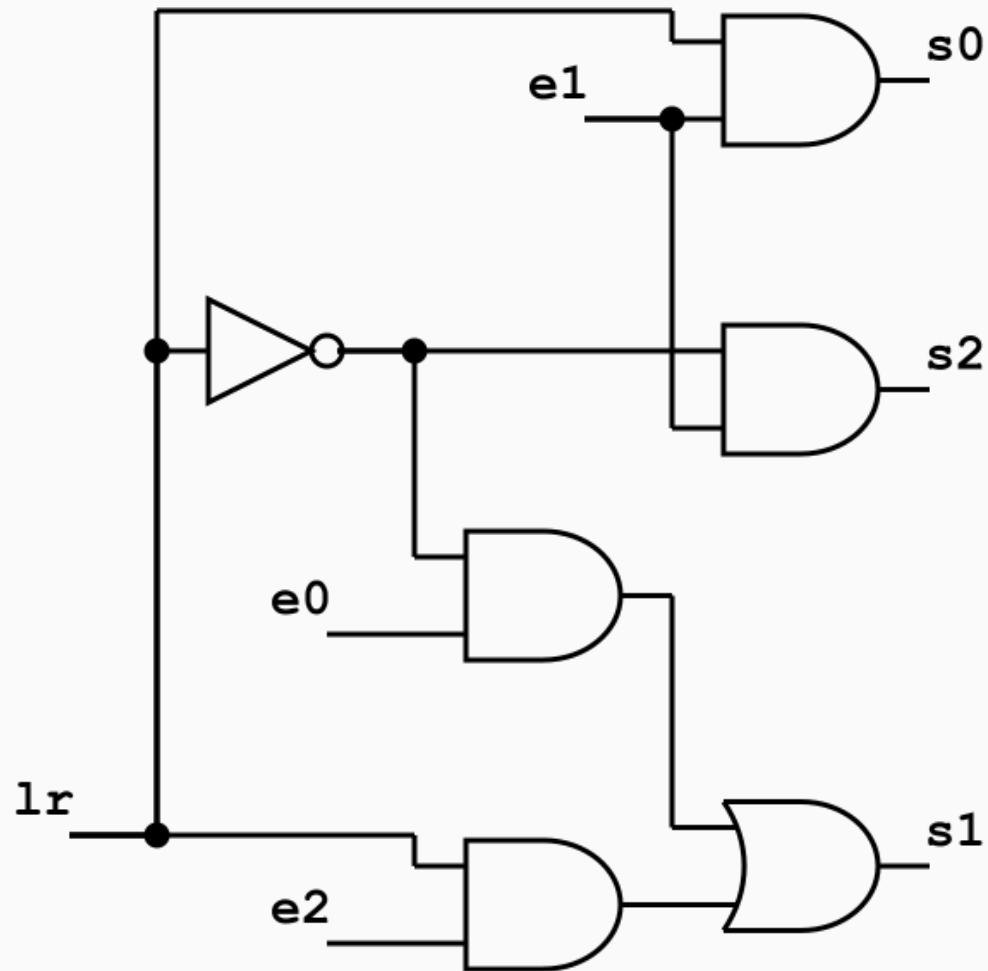
$$s_0 = \begin{cases} 0 & \text{si } lr=0 \\ e_1 & \text{si } lr=1 \end{cases}$$

$$lr \cdot e_1$$

$$s_1 = \begin{cases} e_0 & \text{si } lr=0 \\ e_2 & \text{si } lr=1 \end{cases}$$

$$\overline{lr} \cdot e_0 + lr \cdot e_2$$

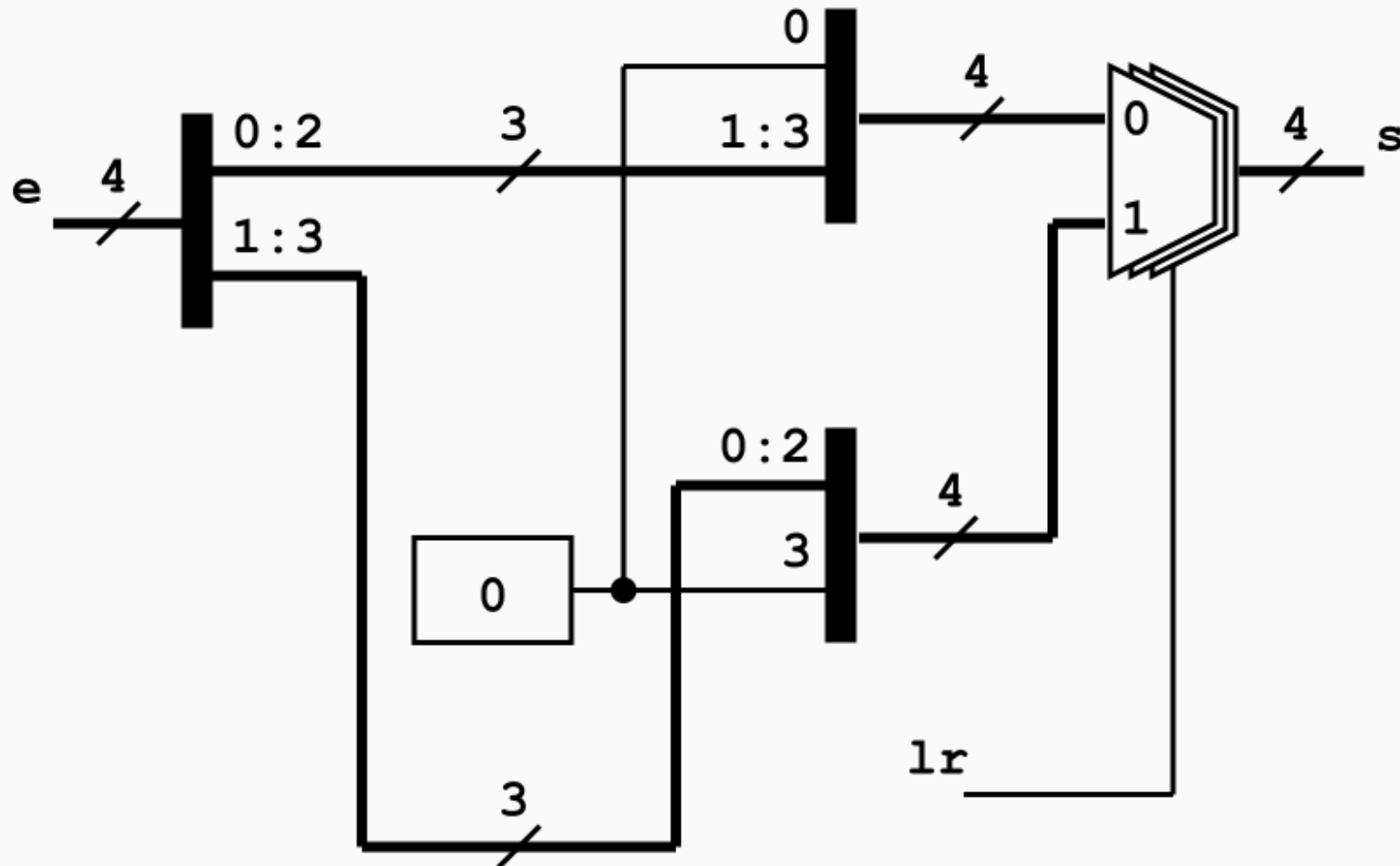
Solución:



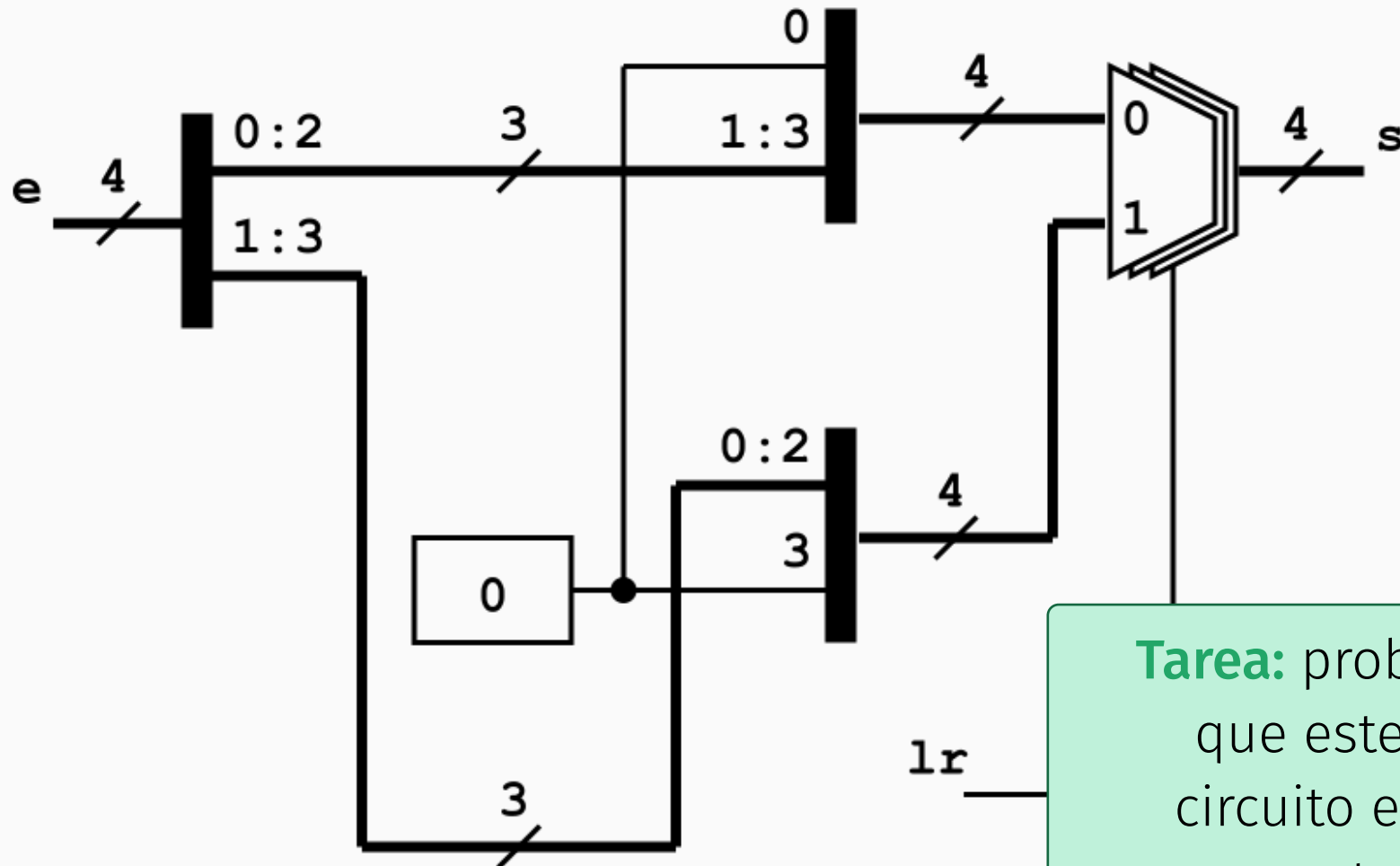
En HDL (SystemVerilog en este caso) podemos trabajar en diferentes niveles de abstracción.

```
module shiftlr #(parameter N = 4) (SV  
    input  logic [N-1:0] e,  
    input  logic          lr, // 0: left, 1: right  
    output logic [N-1:0] s  
);  
  
    always_comb begin  
        if (lr == 1'b0) begin // Shift left  
            s = {e[N-2:0], 1'b0};  
        end else begin // Shift right  
            s = {1'b0, e[N-1:1]};  
        end  
    end  
end  
  
endmodule
```

Al **elaborar** el código, la herramienta yosys genera el siguiente diagrama estructural:

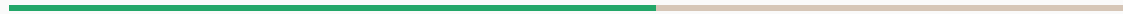


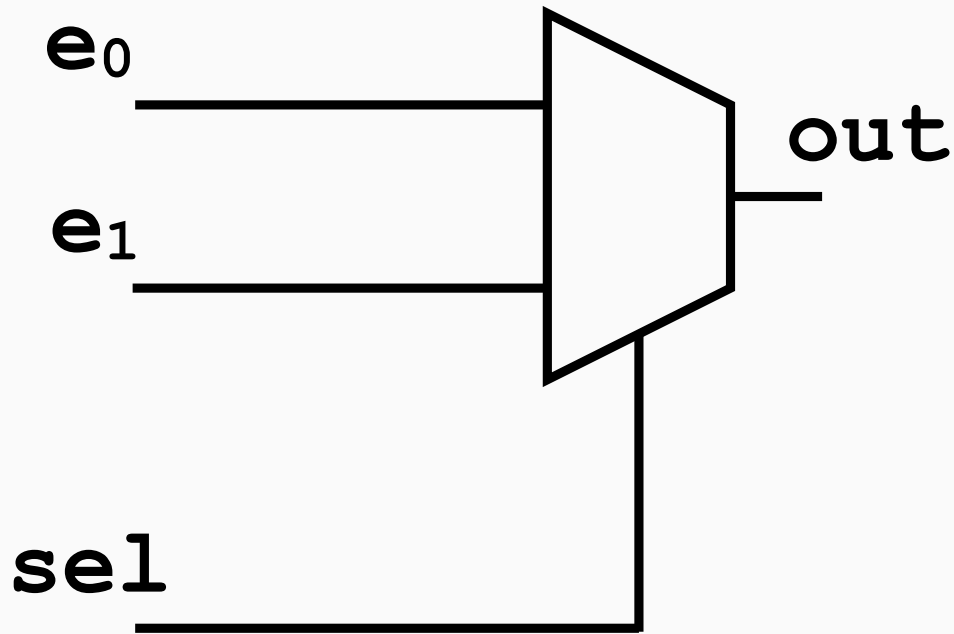
Al **elaborar** el código, la herramienta yosys genera el siguiente diagrama estructural:



Tarea: probar
que este
circuito es
correcto

Multiplexores



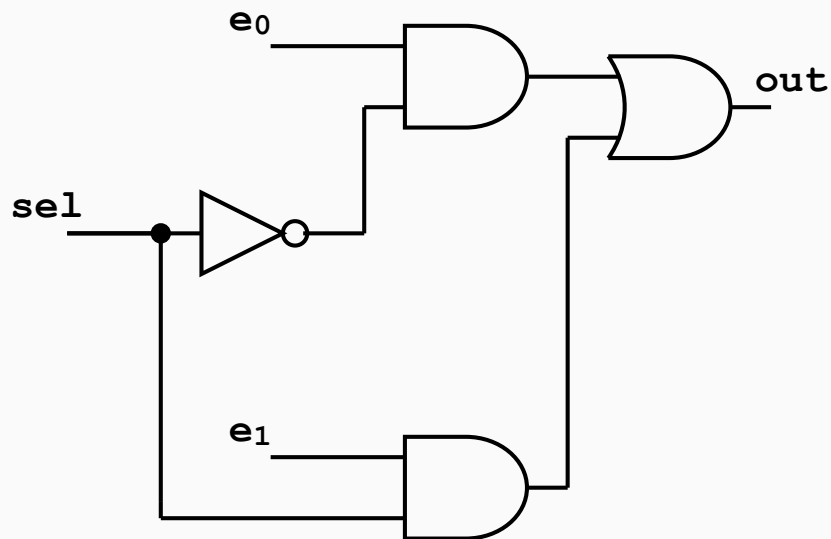


La línea de control **sel** permiten seleccionar una de las entradas e_i , la que corresponderá a la salida **out**.

e_0	e_1	sel	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

$$\text{out} = e_0 \cdot \overline{\text{sel}} + e_1 \cdot \text{sel}$$

Multiplexor 2 a 1

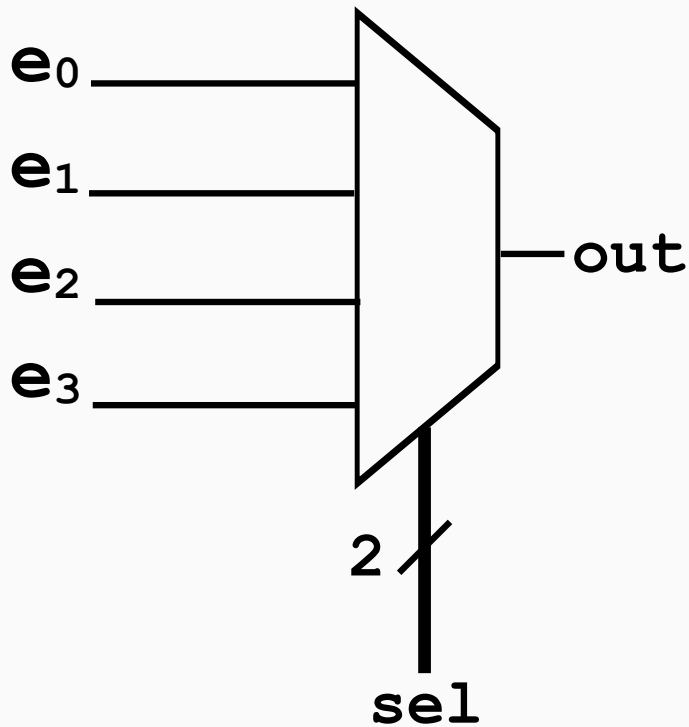


```
module mux2to1 (
    input  logic e_0,
    input  logic e_1,
    input  logic sel,
    output logic out
);

assign out = sel ? e_1 : e_0;

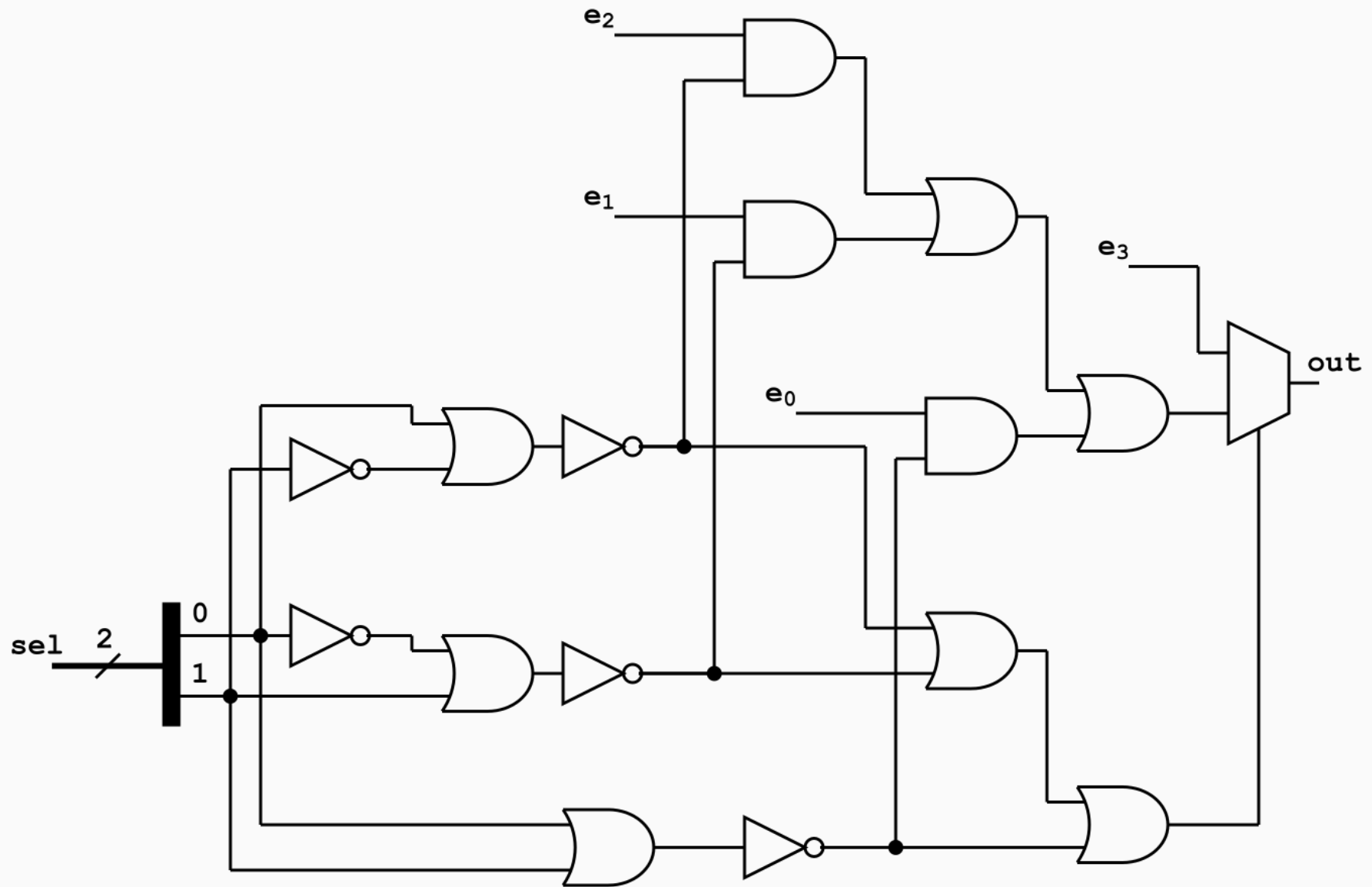
endmodule
```

Multiplexor 4 a 1



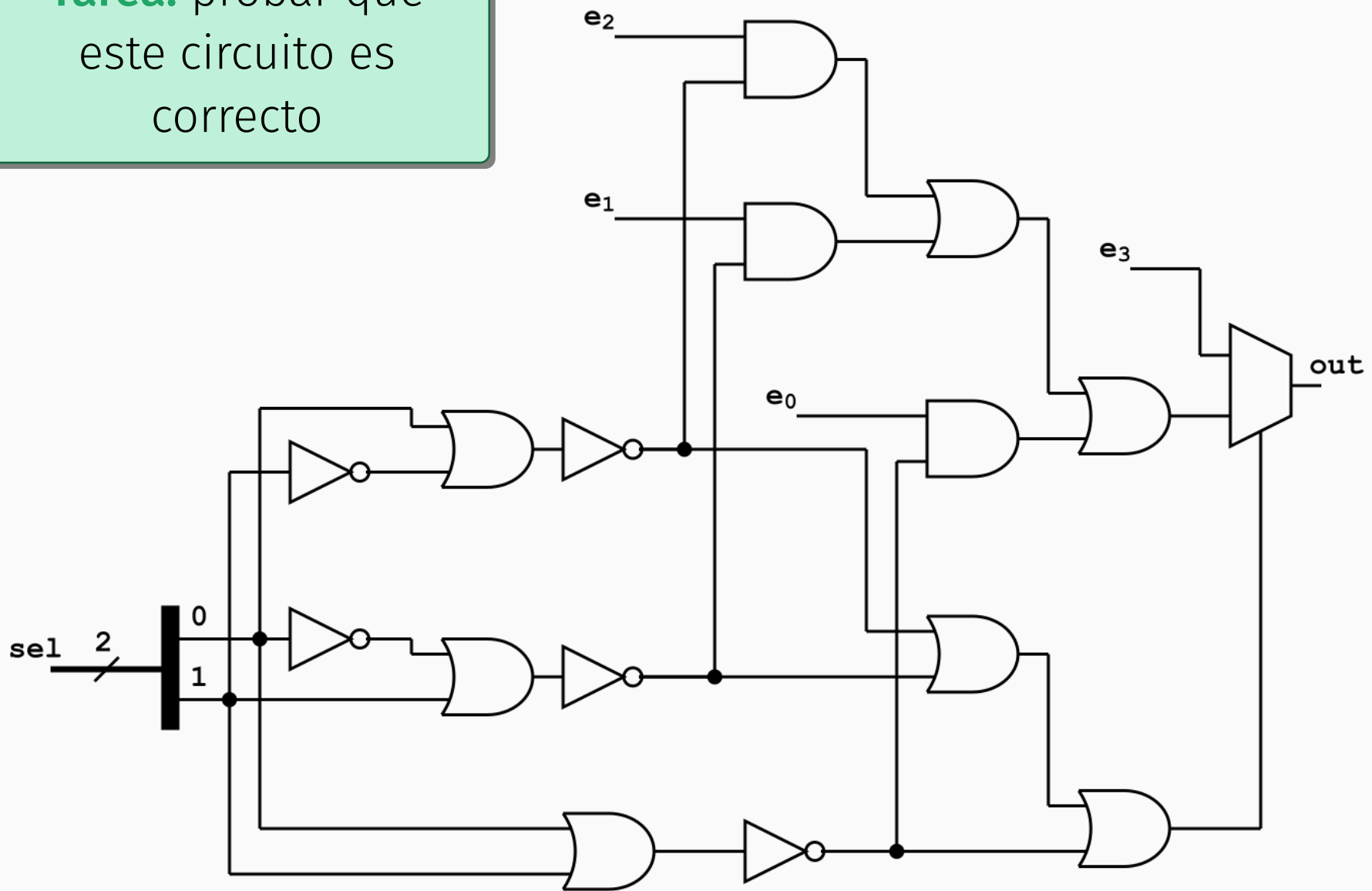
```
module mux4to1 (SV  
    input logic e_0,  
    input logic e_1,  
    input logic e_2,  
    input logic e_3,  
    input logic [1:0] sel,  
    output logic out  
);  
  
    always_comb begin  
        case (sel)  
            2'b00: out = e_0;  
            2'b01: out = e_1;  
            2'b10: out = e_2;  
            default: out = e_3;  
        endcase  
    end  
endmodule
```

Multiplexor 4 a 1

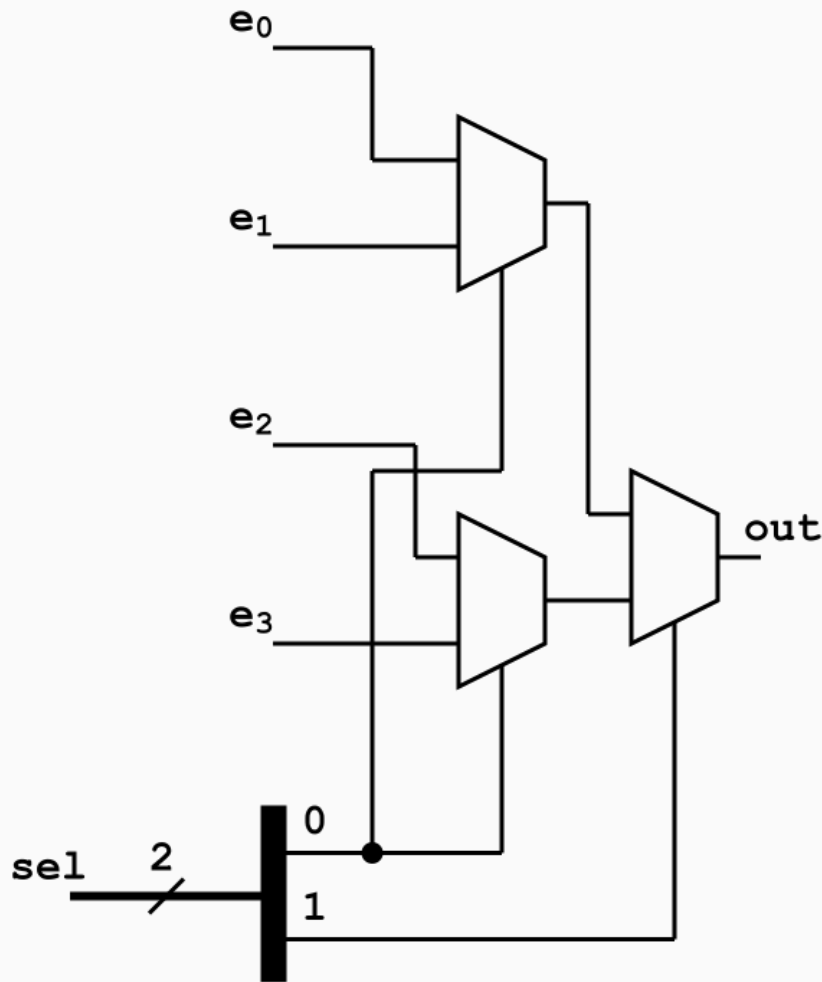


Multiplexor 4 a 1

Tarea: probar que este circuito es correcto



Multiplexor 4 a 1: diseño jerárquico



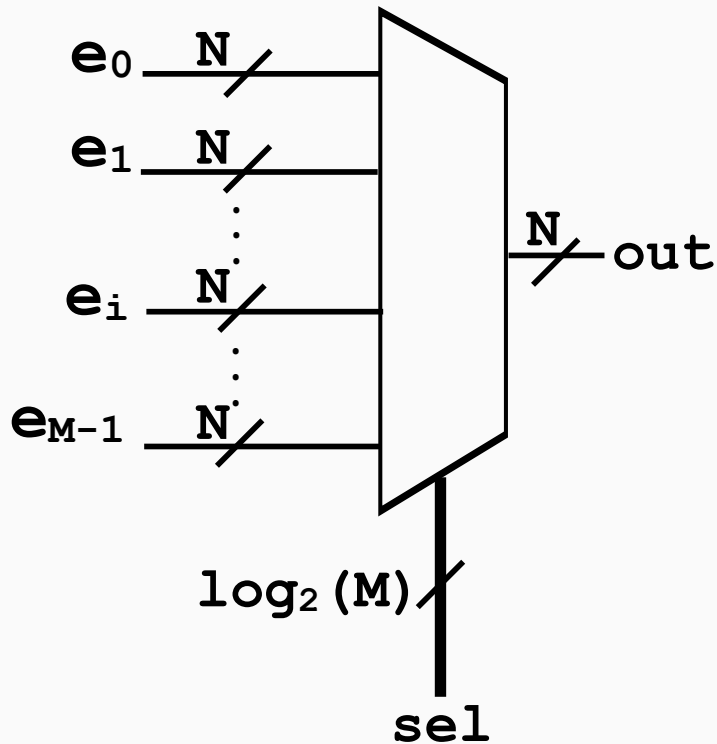
```
// 4-to-1: tree of 2-to-1 muxes SV
module mux4to1_tree (
    input logic e_0,
    input logic e_1,
    input logic e_2,
    input logic e_3,
    input logic [1:0] sel,
    output logic out
);

    logic ab, cd;

    assign ab = sel[0] ? e_1 : e_0;
    assign cd = sel[0] ? e_3 : e_2;
    assign out = sel[1] ? cd : ab;

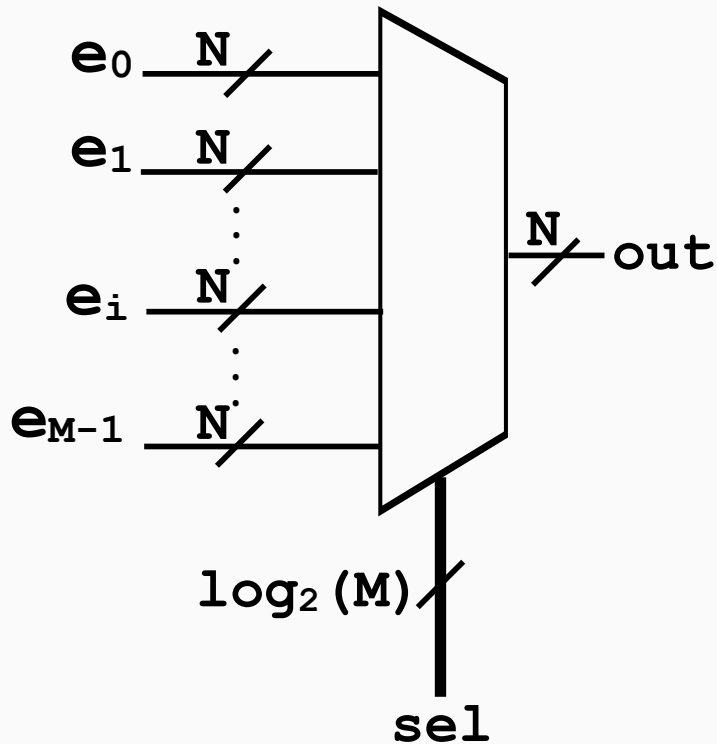
endmodule
```

Multiplexor M a 1: diseño jerárquico



```
module mux #(SV  
    parameter unsigned N = 4,  
    parameter unsigned M = 8  
) (  
    input  logic [N*M-1:0] in,  
    input  logic [$clog2(M)-1:0] sel,  
    output logic [N-1:0] out  
);  
  
    if ((M & (M - 1)) != 0) $error("M debe  
        ser potencia de 2 (M=%0d)", M);  
  
    always_comb begin  
        out = in[sel*N+:N];  
        // in[sel*N + N - 1 : sel*N]  
    end  
endmodule
```


Multiplexor M a 1: diseño jerárquico



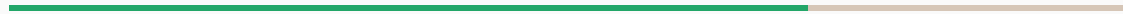
```
module mux2 #(
    parameter unsigned N = 4,
    parameter unsigned M = 8
) (
    input logic [N-1:0] in[M],
    input logic [$clog2(M)-1:0] sel,
    output logic [N-1:0] out
);

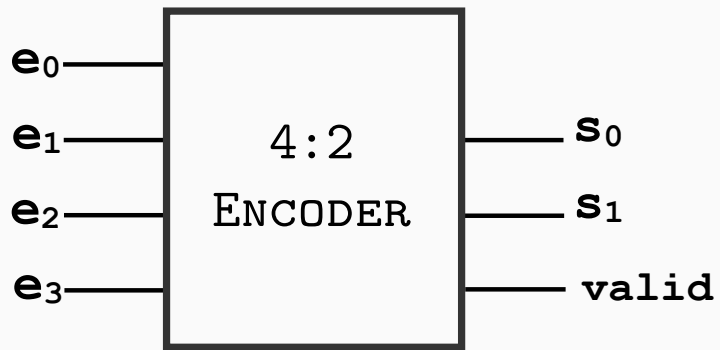
    always_comb begin
        out = in[sel];
    end

endmodule
```

SV

Encoders & Decoders

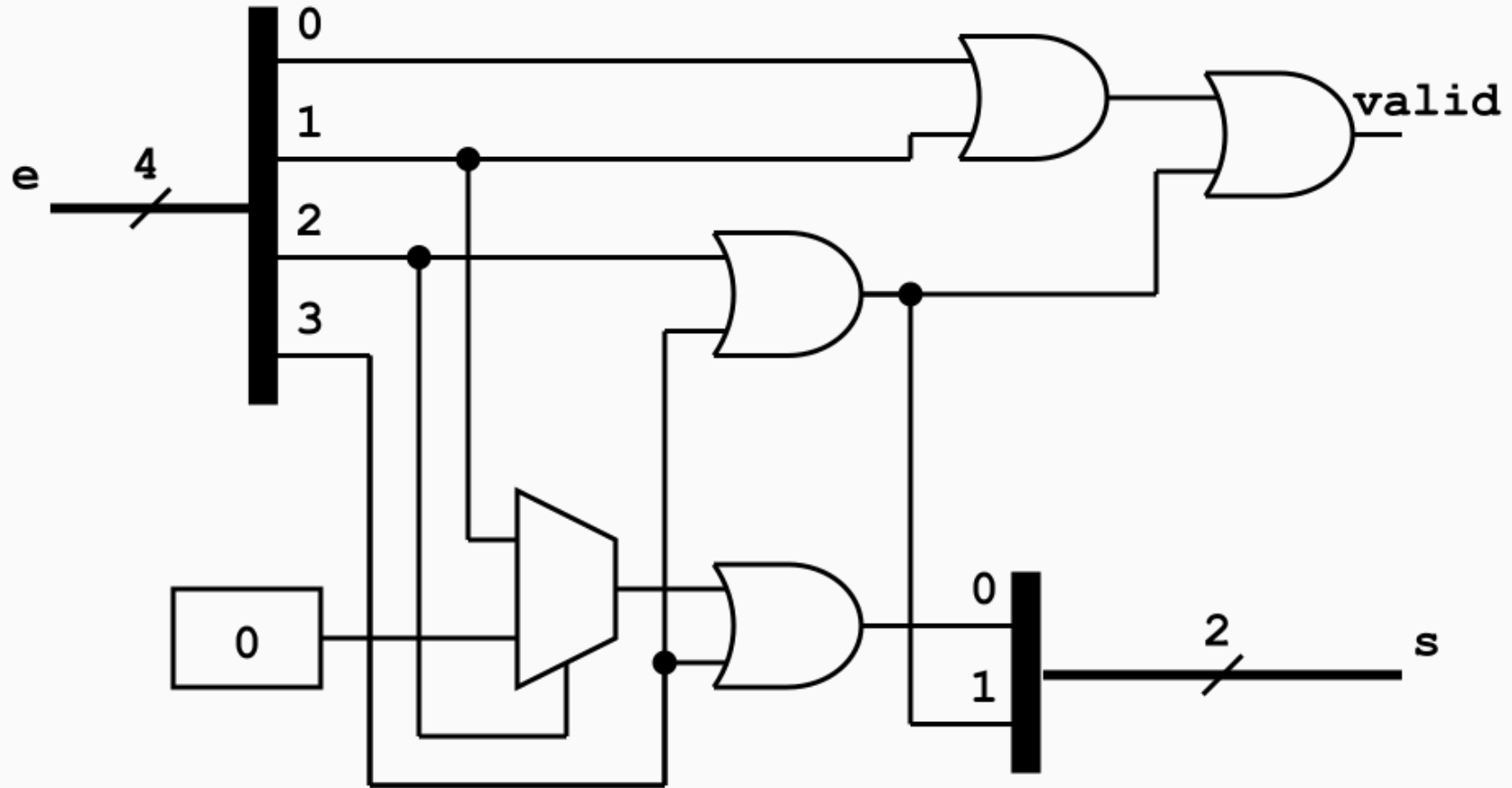




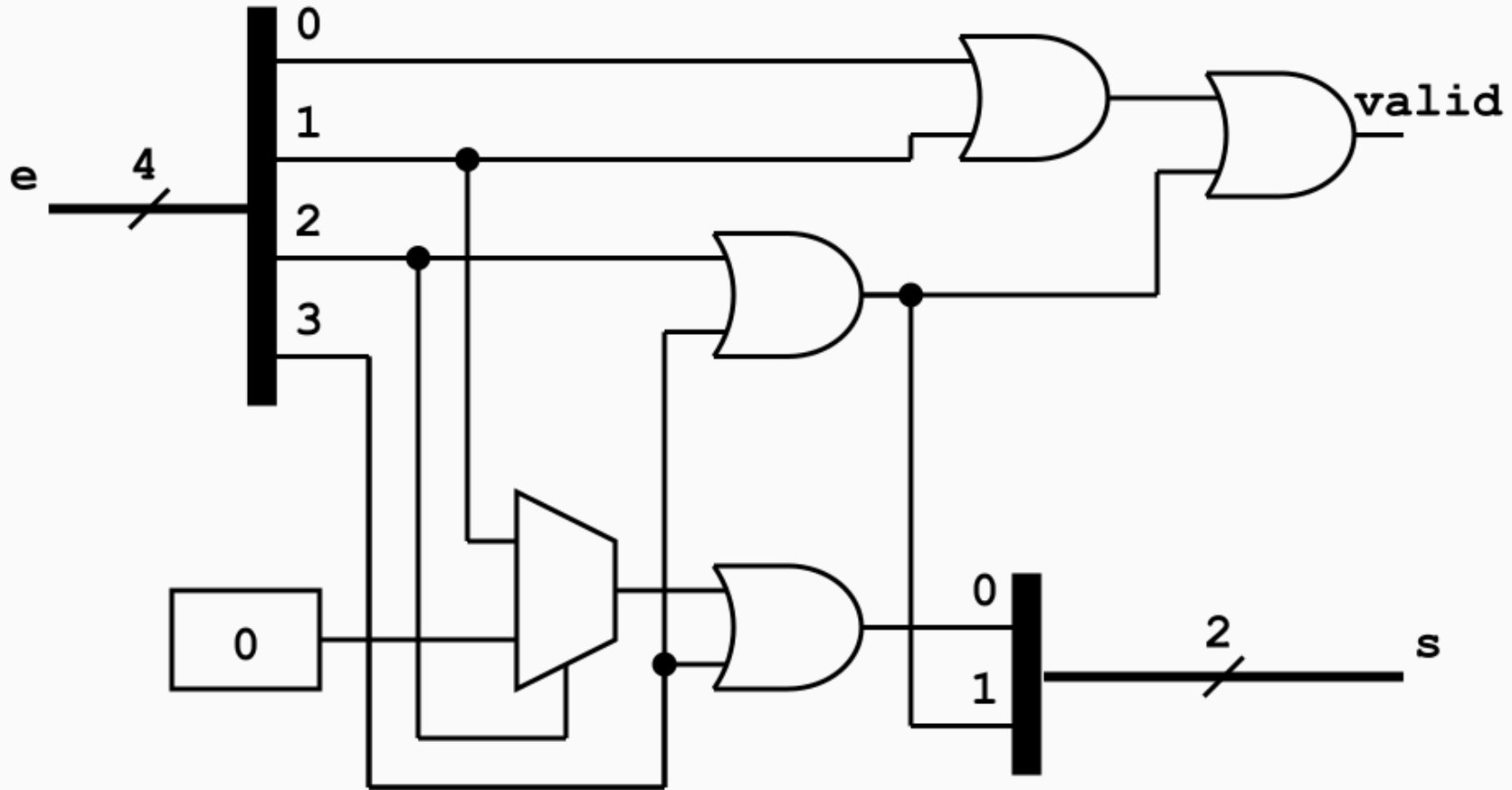
e_3	e_2	e_1	e_0	s	valid
0	0	0	1	0	1
0	0	1	0	1	1
0	1	0	0	2	1
1	0	0	0	3	1
otro				×	0

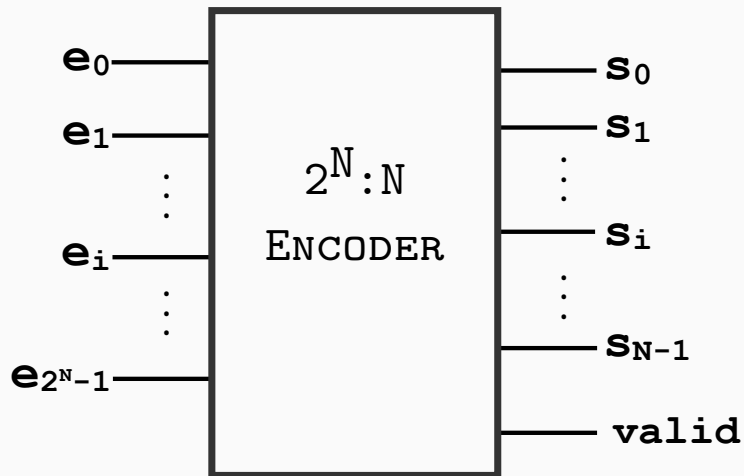
```
module encoder4to2 (
    input logic [3:0] e,
    output logic [1:0] s,
    output logic valid
);

    always_comb begin
        valid = 1'b1;
        case (e)
            4'b0001: s = 2'd0;
            4'b0010: s = 2'd1;
            4'b0100: s = 2'd2;
            4'b1000: s = 2'd3;
            default: begin
                s      = 2'bxx;
                valid = 1'b0;
            end
        endcase
    end
endmodule
```



Tarea: probar que este
circuito es correcto



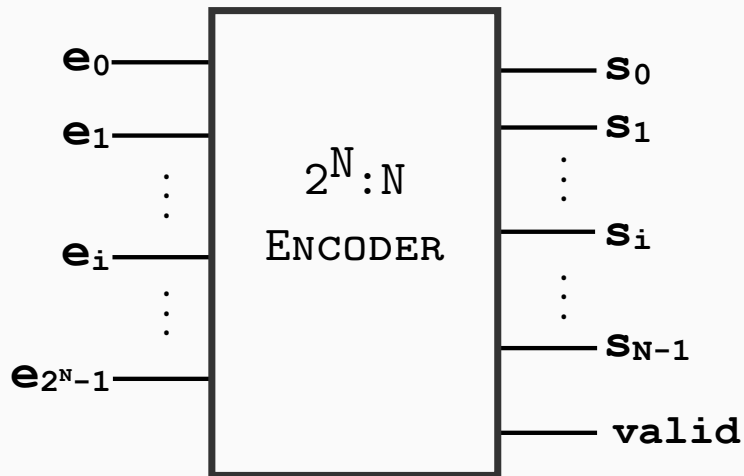


```
module encoder #(
    parameter unsigned N = 4
) (
    input logic [N-1:0] e,
    output logic [$clog2(N)-1:0] s,
    output logic valid
);

    always_comb begin
        s = 0;
        valid = |e;
        for (int unsigned i = 0; i < N; i++)
            begin
                if (e[i]) begin
                    s = i;
                end
            end
        end
    end

endmodule
```

SV

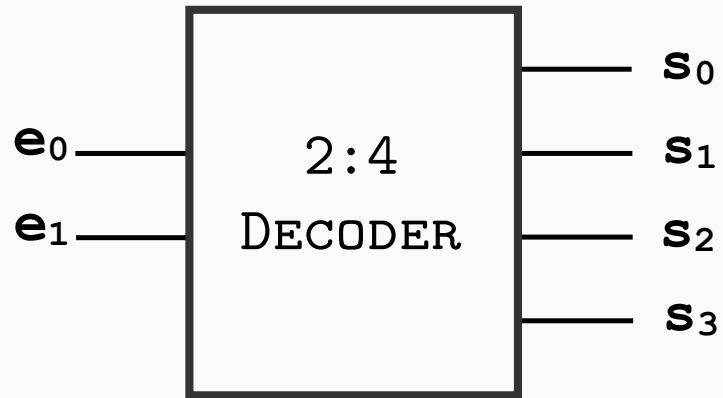


Cuidado: ¿Es lo que queremos?

```
module encoder #(
    parameter unsigned N = 4
) (
    input logic [N-1:0] e,
    output logic [$clog2(N)-1:0] s,
    output logic valid
);

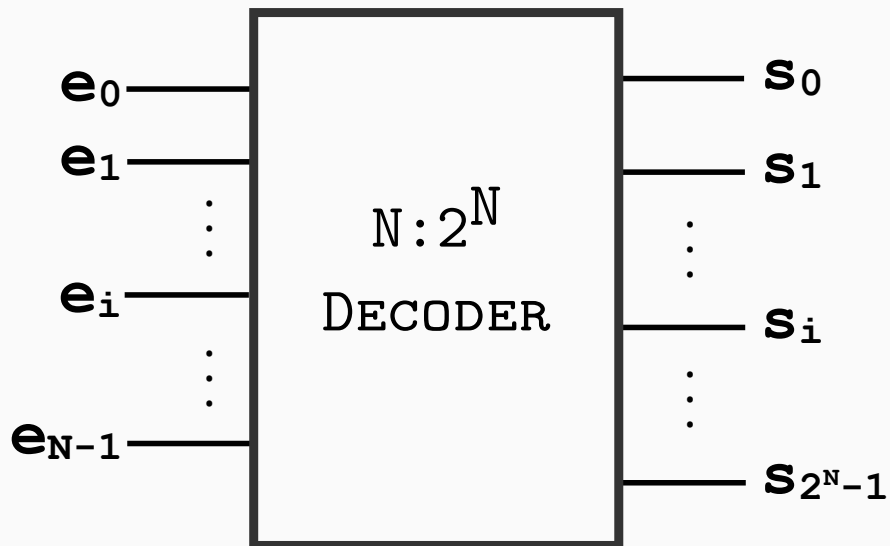
always_comb begin
    s = 0;
    valid = |e;
    for (int unsigned i = 0; i < N; i++)
        begin
            if (e[i]) begin
                s = i;
            end
        end
    end
end

endmodule
```



e_1	e_0	s_3	s_2	s_1	s_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

```
module decoder2to4 (SV  
    input  logic [1:0] e,  
    output logic [3:0] s  
);  
  
    assign s[0] = (e == 2'b00);  
    assign s[1] = (e == 2'b01);  
    assign s[2] = (e == 2'b10);  
    assign s[3] = (e == 2'b11);  
  
endmodule
```

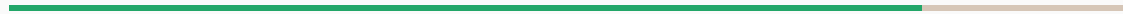
```
module decoder #(
    parameter unsigned N = 2
) (
    input logic [N-1:0] e,
    output logic [2*N-1:0] s
);

    always_comb begin
        s = '0;
        s[e] = 1'b1;
    end

endmodule
```

SV

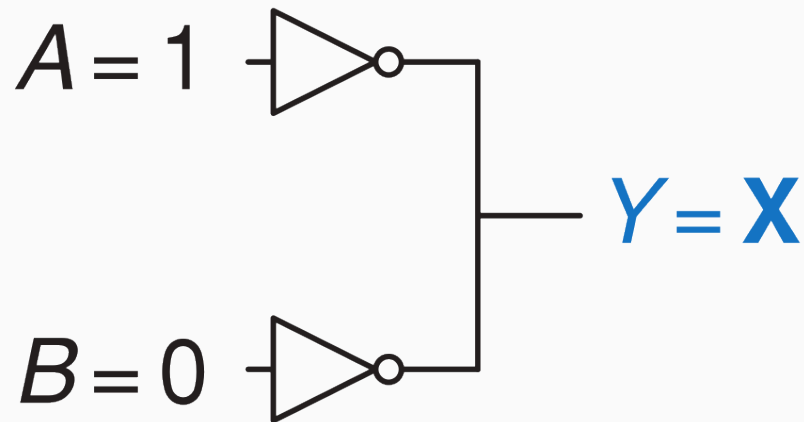
Valores especiales: X y Z



El símbolo **X** representa un valor ilegal, que puede aparecer en la simulación de un circuito digital cuando:

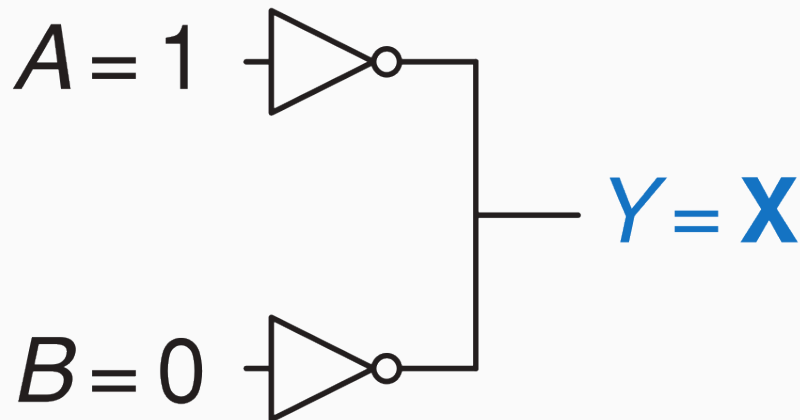
El símbolo **X** representa un valor ilegal, que puede aparecer en la simulación de un circuito digital cuando:

- una señal no ha sido inicializada, o



El símbolo **X** representa un valor ilegal, que puede aparecer en la simulación de un circuito digital cuando:

- una señal no ha sido inicializada, o
- una señal es el resultado de una operación ilegal



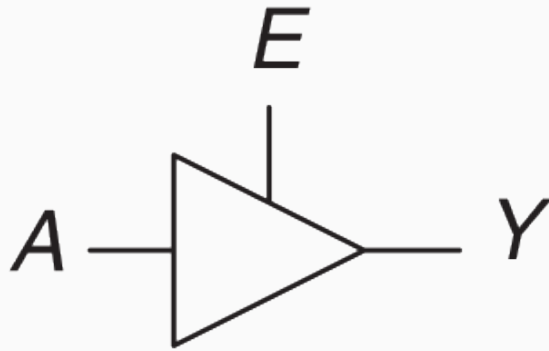
```
module decoder_buggy (
    input logic [1:0] sel,
    output logic [3:0] s
);
    always_comb begin
        case (sel)
            2'b00: s = 4'b0001;
            2'b01: s = 4'b0010;
            2'b10: s = 4'b0100;
            // falta 2'b11
        endcase
    end
endmodule
```

Z: Alta impedancia

El símbolo **Z** representa un valor de alta impedancia, lo que significa que la señal no está siendo conducida por ningún circuito. Se implementa con un **buffer tri-state**:

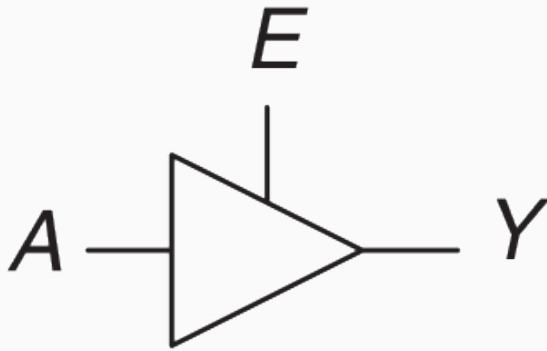
Z: Alta impedancia

El símbolo **Z** representa un valor de alta impedancia, lo que significa que la señal no está siendo conducida por ningún circuito. Se implementa con un **buffer tri-state**:



Z: Alta impedancia

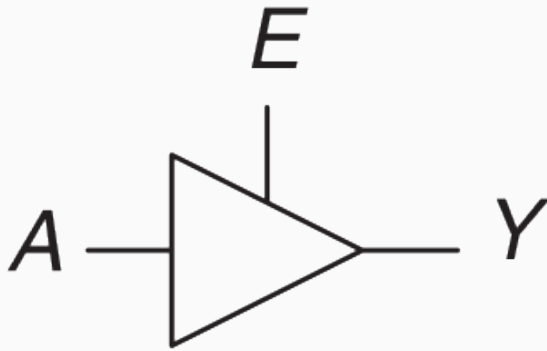
El símbolo **Z** representa un valor de alta impedancia, lo que significa que la señal no está siendo conducida por ningún circuito. Se implementa con un **buffer tri-state**:



<i>E</i>	<i>A</i>	<i>Y</i>
0	0	<i>Z</i>
0	1	<i>Z</i>
1	0	0
1	1	1

Z: Alta impedancia

El símbolo **Z** representa un valor de alta impedancia, lo que significa que la señal no está siendo conducida por ningún circuito. Se implementa con un **buffer tri-state**:

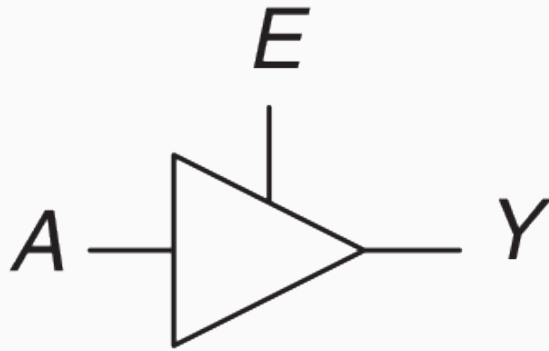


<i>E</i>	<i>A</i>	<i>Y</i>
0	0	<i>Z</i>
0	1	<i>Z</i>
1	0	0
1	1	1

Z es también llamado *high Z*, *floating* o *tri-state*.

Z: Alta impedancia

El símbolo **Z** representa un valor de alta impedancia, lo que significa que la señal no está siendo conducida por ningún circuito. Se implementa con un **buffer tri-state**:

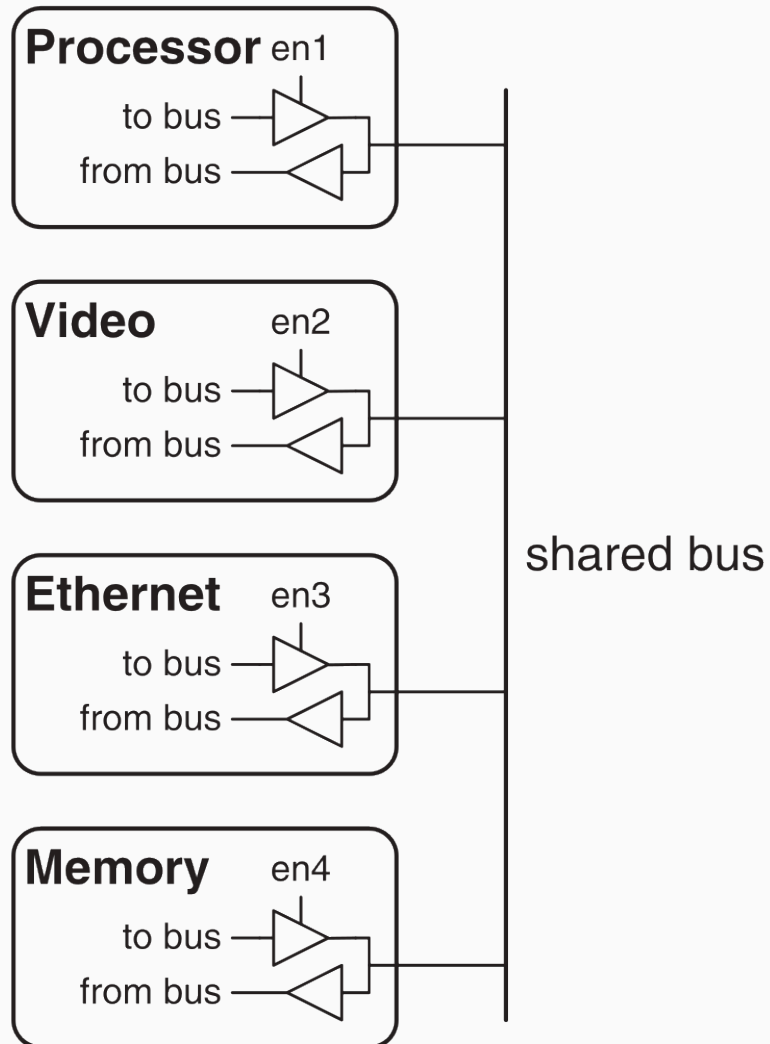


E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

Z es también llamado *high Z*, *floating* o *tri-state*.

misconception: Una señal desconectada o “al aire” no toma un 0 lógico. Puede tomar cualquier valor

Z: Alta impedancia

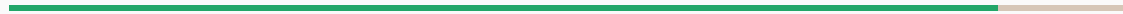


```
module tristate_buffer (
    input logic e,
    input logic to_bus,
    output logic from_bus,
    inout tri logic bus
);

    assign bus = e ? to_bus : 1'bz;
    assign from_bus = bus;

endmodule
```

Timing



¡Las compuertas no son instantáneas!

¡Las compuertas no son instantáneas!

- **Delay de propagación:**

Tiempo máximo hasta que la salida se estabiliza.

¡Las compuertas no son instantáneas!

- **Delay de propagación:**

Tiempo máximo hasta que la salida se estabiliza.

- **Delay de contaminación:**

Tiempo mínimo hasta que la salida comienza a cambiar (a.k.a. tiempo de transición de salida).

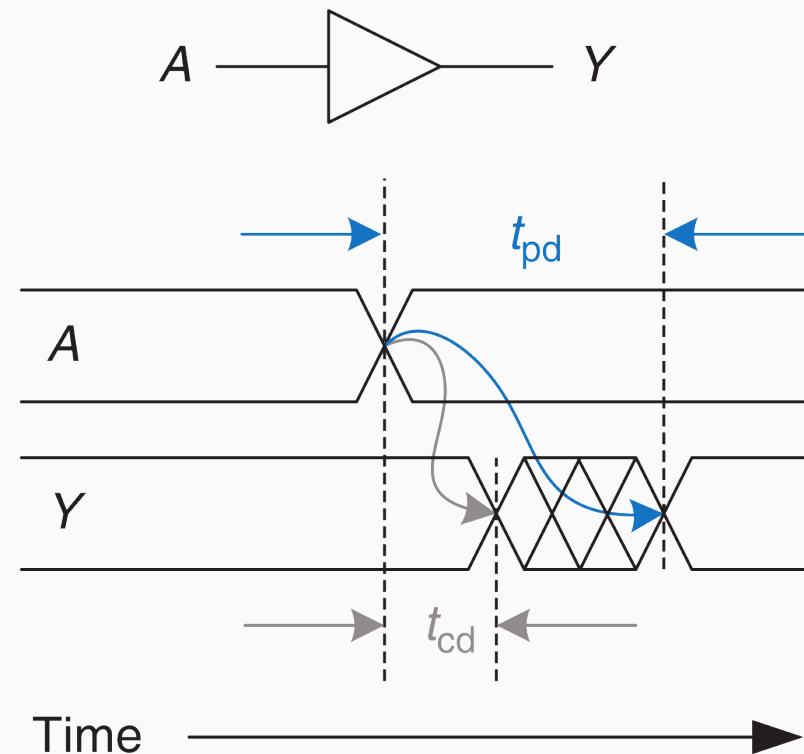
¡Las compuertas no son instantáneas!

- **Delay de propagación:**

Tiempo máximo hasta que la salida se estabiliza.

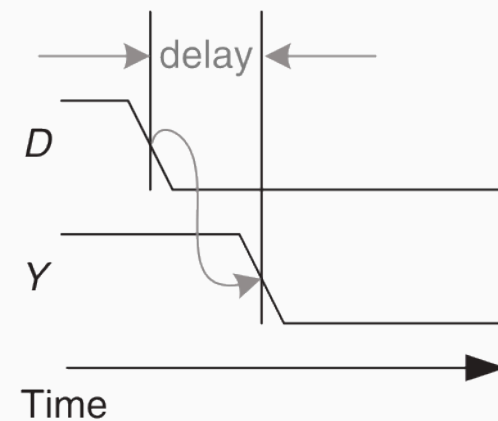
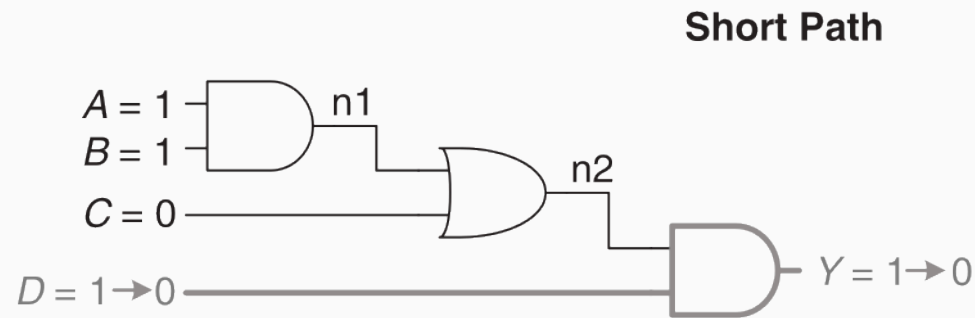
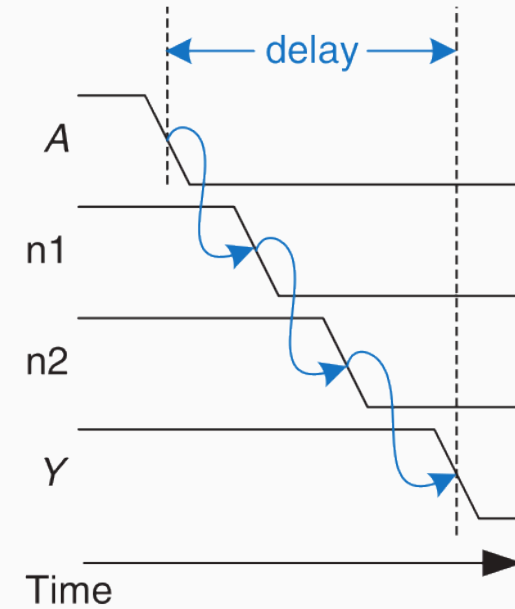
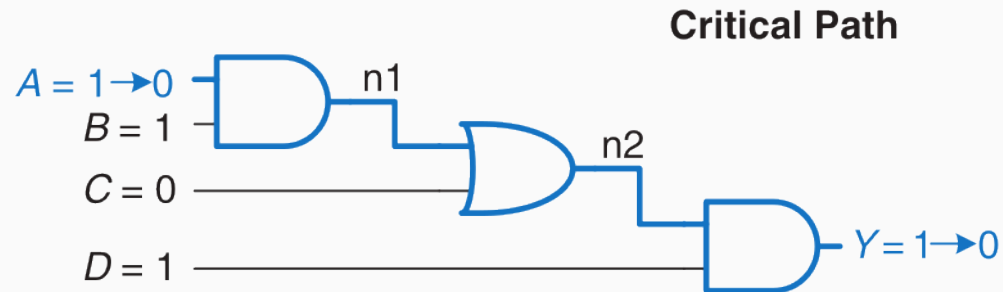
- **Delay de contaminación:**

Tiempo mínimo hasta que la salida comienza a cambiar (a.k.a. tiempo de transición de salida).

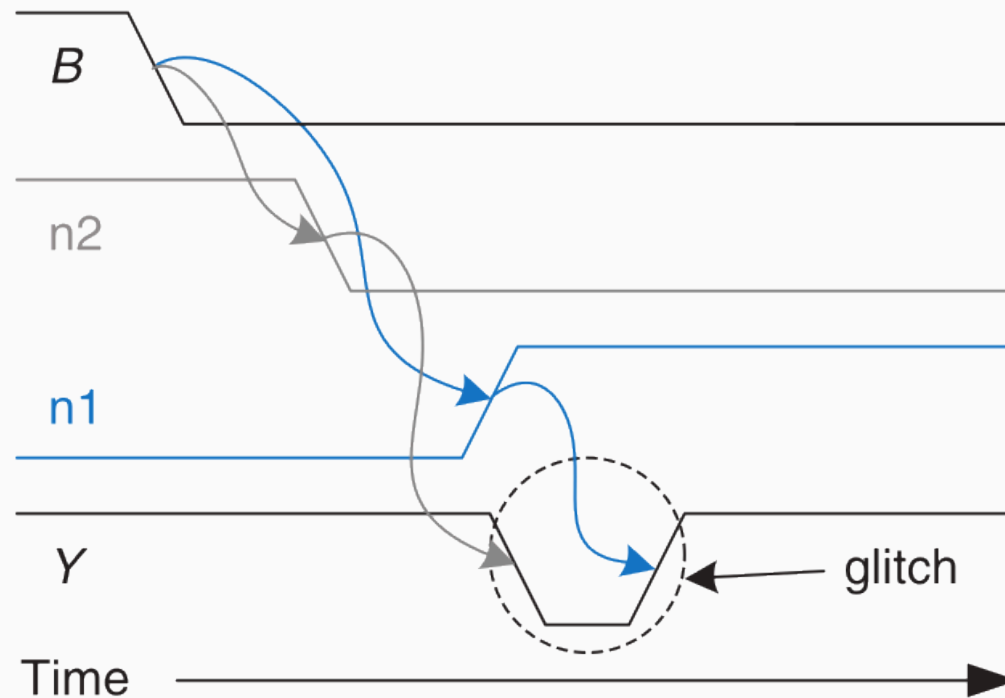
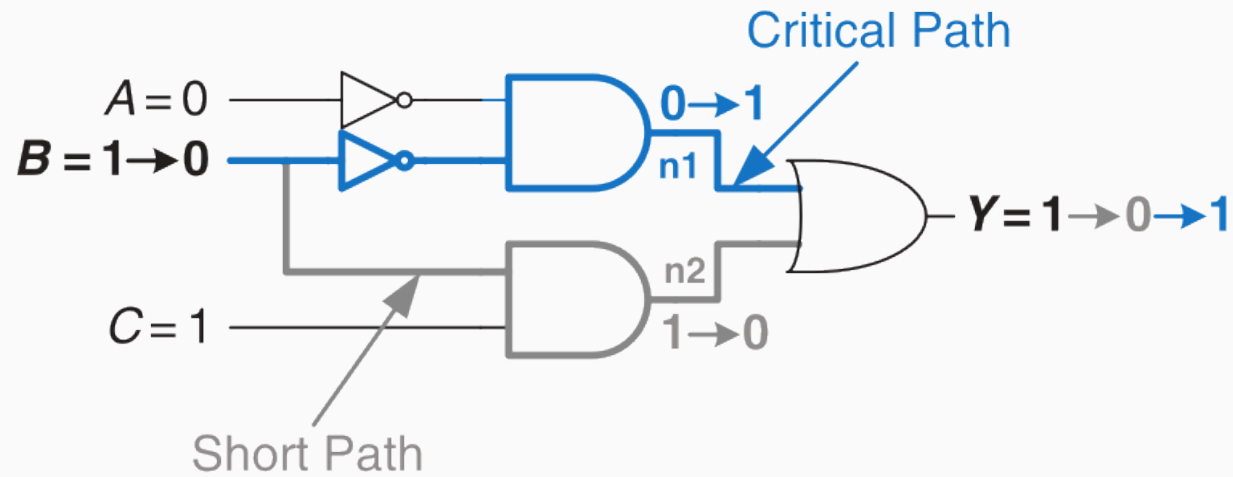


¿Cómo impacta esto en los sistemas que diseñamos?

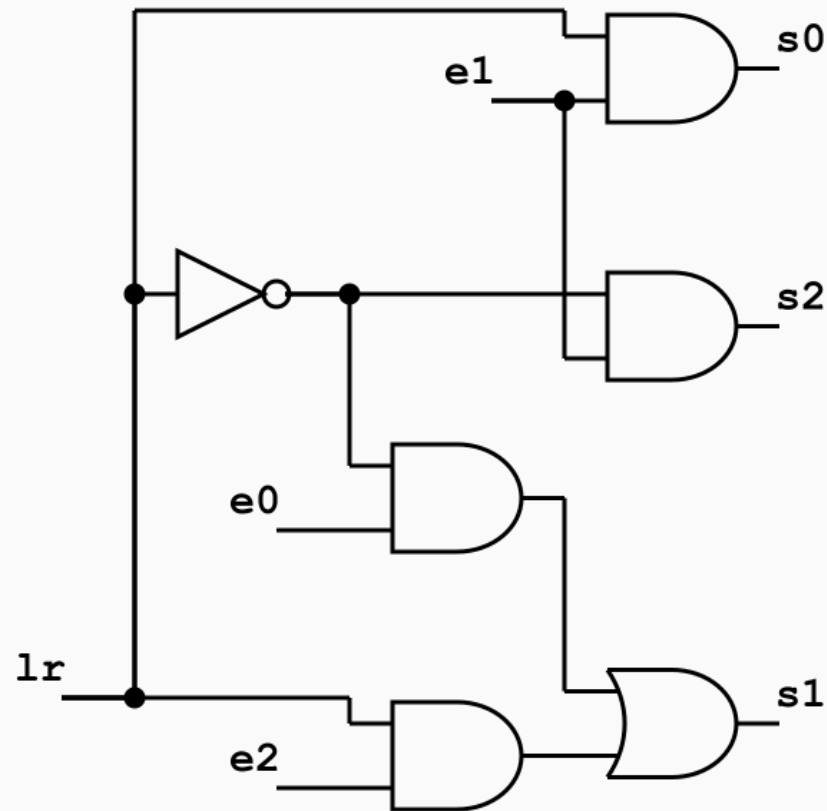
¿Cómo impacta esto en los sistemas que diseñamos?



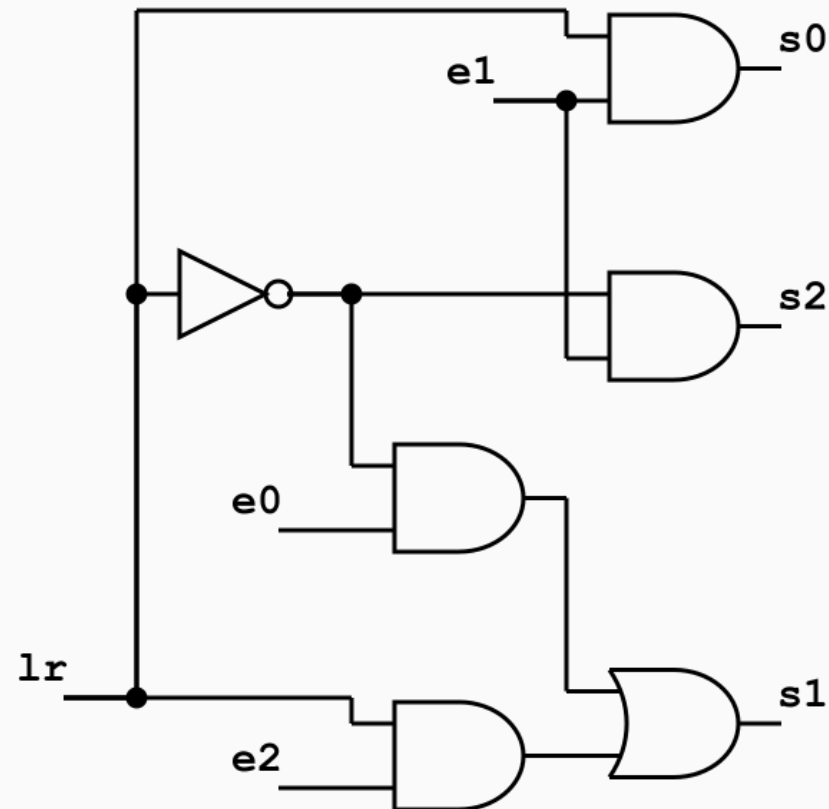
Glitches... oh my!



Revisitemos nuestro Shift LR:

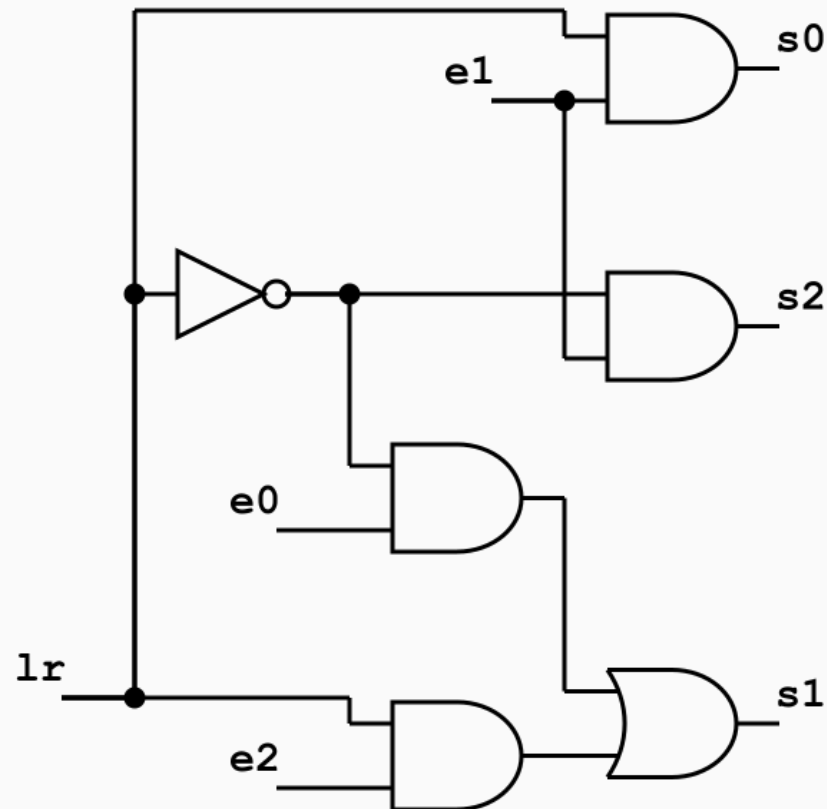


Revisitemos nuestro Shift LR:

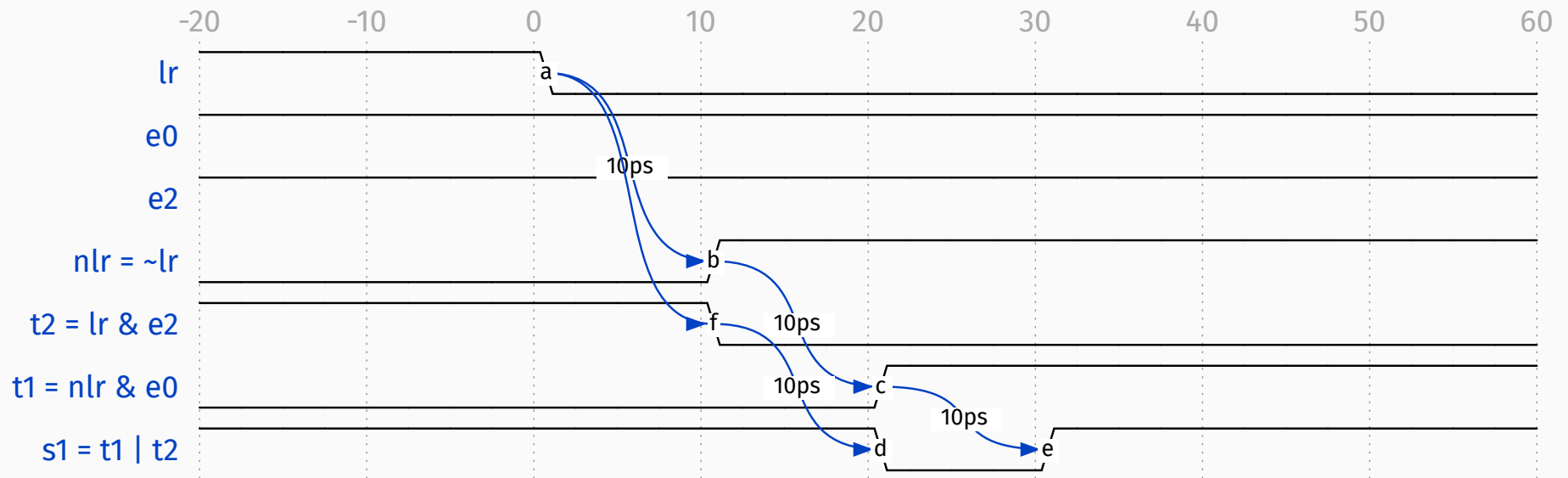
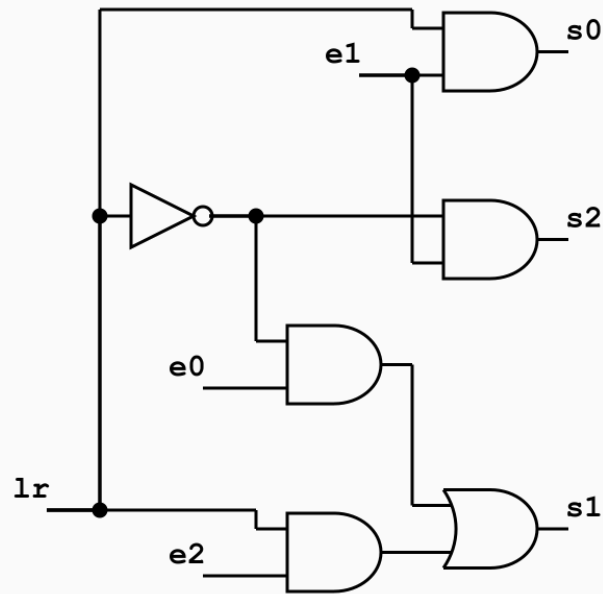


- ¿Cuál es el camino crítico?

Revisitemos nuestro Shift LR:



- ¿Cuál es el camino crítico?
- ¿Hay glitches?



Camino crítico: $lr \rightarrow \overline{lr} \rightarrow t_1 \rightarrow s_1$ (3 compuertas, 30ps).

Hay glitches: s_1 “parpadea” a 0 entre los 20ps y los 30ps antes de estabilizarse.

¿Cuál es el **mínimo tiempo** que se debe esperar para leer un resultado válido de su salida?

Camino crítico: $lr \rightarrow \overline{lr} \rightarrow t_1 \rightarrow s_1$ (3 compuertas, 30ps).

Hay glitches: s_1 “parpadea” a 0 entre los 20ps y los 30ps antes de estabilizarse.

¿Cuál es el **mínimo tiempo** que se debe esperar para leer un resultado válido de su salida?

- En este caso debemos esperar al menos $3 \cdot 10 \text{ ps} = 30 \text{ ps}$ para poder leer la salida.

Camino crítico: $l_r \rightarrow \bar{l}_r \rightarrow t_1 \rightarrow s_1$ (3 compuertas, 30ps).

Hay glitches: s_1 “parpadea” a 0 entre los 20ps y los 30ps antes de estabilizarse.

¿Cuál es el **mínimo tiempo** que se debe esperar para leer un resultado válido de su salida?

- En este caso debemos esperar al menos $3 \cdot 10 \text{ ps} = 30 \text{ ps}$ para poder leer la salida.

¿Cómo enfrentamos (en parte) este problema?

Secuenciales... (en el próximo episodio)

¿Preguntas?
